SENA

SERVICIO NACIONAL
DE APRENDIZAJE

BIENVENIDOS AL
MANUAL
TÉCNICO



CENTRO
SIMULACIÓN EN SALUD

APRENDER • PRACTICAR • CUIDAR

ANGIE GOMEZ • JOHAN GARCIA

CAPÍTULO 1. GENERALIDADES DEL SISTEMA 12
1.1. Descripción del Centro de Simulación en Salud del SENA 12
1.2. Descripción del sistema de información 12
1.3. Antecedentes y necesidad del desarrollo 13
1.4. Objetivos del sistema 13
1.5. Alcance funcional del frontend 14
1.6. Características principales 14
1.7. Tipos de usuarios 15
1.8. Roles y permisos 15
1.9. Restricciones técnicas 16
CAPÍTULO 2. FUNDAMENTOS TECNOLÓGICOS 17
2.1. Introducción al desarrollo frontend 17
2.2. ¿Qué es React? 17
2.3. Características de React 17
2.4. Arquitectura basada en componentes 18
2.5. JavaScript y JSX 18
2.6. HTML5 y CSS3 19
2.7. Vite como herramienta de desarrollo 19
2.8. React Router DOM 19
2.9. Bibliotecas y dependencias utilizadas 20
2.10. Ventajas de las tecnologías implementadas 20
CAPÍTULO 3. REQUISITOS DEL ENTORNO DE DESARROLLO 21
3.1. Requisitos de hardware 21
3.2. Requisitos de software 21
3.3. Sistema operativo compatible 21
3.4. Instalación de Node.js 22
3.5. Instalación de Visual Studio Code 22

3.6. Instalación de Git 23

3.7. Configuración del entorno 23

3.8. Verificación de herramientas 23

3.9. Preparación del proyecto 24

CAPÍTULO 4. INSTALACIÓN Y CONFIGURACIÓN DEL PROYECTO 25

4.1. Estructura inicial del proyecto 25

4.2. Archivo package.json 25

4.3. Dependencias principales 25

4.4. Dependencias de desarrollo 26

4.5. Instalación de paquetes 26

4.6. Configuración de Vite 26

4.7. Configuración del punto de entrada 27

4.8. Ejecución del servidor local 27

4.9. Compilación del proyecto 27

4.10. Visualización de errores de ejecución 28

CAPÍTULO 5. ARQUITECTURA Y ESTRUCTURA DEL FRONTEND 29

5.1. Descripción general de la arquitectura 29

5.2. Organización de carpetas 29

5.3. Carpeta Components 30

5.4. Carpeta Pages 30

5.5. Carpeta Context 30

5.6. Carpeta Layouts 31

5.7. Carpeta Practicas 31

5.8. Carpeta Guias 31

5.9. Carpeta de recursos e imágenes 32

5.10. Archivo App.jsx 32

5.11. Archivo main.jsx 32

5.12. Relación entre componentes 33

5.13. Flujo de navegación 33

CAPÍTULO 6. COMPONENTES DE REACT 34

6.1. Concepto de componente 34

6.2. Componentes funcionales 34

6.3. Propiedades (Props) 35

6.4. Estado interno 35

6.5. Eventos 36

6.6. Renderizado condicional 36

6.7. Listas y claves 37

6.8. Reutilización de componentes 37

6.9. Comunicación entre componentes 37

6.10. Buenas prácticas de desarrollo 38

CAPÍTULO 7. GESTIÓN DE ESTADOS Y CONTEXTO 39

7.1. Concepto de estado 39

7.2. Hook useState 39

7.3. Hook useEffect 40

7.4. Context API 40

7.5. AuthContext 40

7.6. Proveedor de autenticación 41

7.7. Consumo del contexto 41

7.8. Actualización de información 41

7.9. Manejo de sesiones 42

7.10. Consideraciones de persistencia 42

CAPÍTULO 8. SISTEMA DE RUTAS Y NAVEGACIÓN 43

8.1. Introducción a React Router DOM 43

8.2. Configuración de BrowserRouter 43

8.3. Definición de rutas 43
8.4. Navegación entre páginas 44
8.5. Rutas públicas 44
8.6. Rutas privadas 44
8.7. Componente PrivateRoute 44
8.8. Redirecciones 45
8.9. Página no encontrada 45
8.10. Pruebas de navegación 45
CAPÍTULO 9. DISEÑO E INTERFAZ DE USUARIO 46
9.1. Principios de diseño frontend 46
9.2. Identidad visual institucional 46
9.3. Distribución de la interfaz 46
9.4. DashboardLayout 47
9.5. Barra lateral (Sidebar) 47
9.6. Barra superior (Navbar) 47
9.7. Diseño de formularios 47
9.8. Tablas y listados 48
9.9. Ventanas y paneles emergentes 48
9.10. Diseño adaptable 48
9.11. Accesibilidad y usabilidad 49
CAPÍTULO 10. MÓDULO DE AUTENTICACIÓN Y USUARIOS 50
10.1. Introducción al módulo 50
10.2. Inicio de sesión 50
10.3. Registro de usuarios 50
10.4. Recuperación de contraseña 51
10.5. Gestión del perfil 51
10.6. Administración de usuarios 51

10.7. Roles del sistema 52
10.8. Estados activo e inactivo 52
10.9. Validación de formularios 52
10.10. Almacenamiento de información de usuarios 53
CAPÍTULO 11. MÓDULO DE AMBIENTES 54
11.1. Descripción funcional 54
11.2. Estructura del módulo 54
11.3. Visualización de ambientes 54
11.4. Registro y consulta 54
11.5. Reservas de ambientes 55
11.6. Validación de disponibilidad 55
11.7. Almacenamiento de reservas 55
11.8. Manejo de novedades 56
CAPÍTULO 12. MÓDULO DE EQUIPOS 57
12.1. Descripción funcional 57
12.2. Registro de equipos 57
12.3. Consulta de equipos 57
12.4. Estados de los equipos 57
12.5. Control de disponibilidad 58
12.6. Registro de uso 58
12.7. Gestión de novedades 58
12.8. Validación de información 58
CAPÍTULO 13. MÓDULO DE INVENTARIO 59
13.1. Descripción general 59
13.2. Estructura del inventario 59
13.3. Registro de insumos 59
13.4. Consulta y búsqueda 59

13.5. Actualización de existencias 60
13.6. Control de stock mínimo 60
13.7. Control de fechas de vencimiento 60
13.8. Ubicación de insumos 61
13.9. Movimientos de inventario 61
13.10. Estados de disponibilidad 61
13.11. Validaciones 61
13.12. Persistencia de datos 62
CAPÍTULO 14. MÓDULO DE CRONOGRAMA Y RESERVAS 63
14.1. Descripción del módulo 63
14.2. Organización de actividades 63
14.3. Visualización del cronograma 63
14.4. Registro de actividades 63
14.5. Reservas de ambientes 64
14.6. Control de fechas y horarios 64
14.7. Validación de información 64
CAPÍTULO 15. MÓDULO DE SIMULACIONES Y PRÁCTICAS 65
15.1. Descripción del módulo 65
15.2. Organización de simulaciones 65
15.3. Gestión de prácticas 65
15.4. Recursos asociados 65
15.5. Navegación de contenidos 66
15.6. Registro de información 66
15.7. Relación con los ambientes y equipos 66
CAPÍTULO 16. MÓDULO DE GUÍAS Y EVALUACIONES 67
16.1. Gestión de guías 67
16.2. Organización de contenidos 67

16.3. Visualización de guías 67
16.4. Gestión de evaluaciones 67
16.5. Registro de resultados 68
16.6. Validación de respuestas 68
16.7. Presentación de resultados 68
CAPÍTULO 17. MÓDULOS DE REPORTES Y NOTIFICACIONES 69
17.1. Descripción de reportes 69
17.2. Generación de reportes 69
17.3. Exportación de información 69
17.4. Generación de documentos PDF 69
17.5. Notificaciones del sistema 70
17.6. Panel de notificaciones 70
17.7. Preferencias de notificación 70
CAPÍTULO 18. ASISTENTE VIRTUAL DEL SISTEMA 71
18.1. Descripción del asistente 71
18.2. Componente IAButton 71
18.3. Estructura del componente 71
18.4. Interfaz de interacción 72
18.5. Procesamiento de consultas 72
18.6. Consultas sobre usuarios 72
18.7. Consultas sobre inventario 72
18.8. Consultas sobre equipos y ambientes 73
18.9. Limitaciones del asistente 73
18.10. Mejoras futuras 73
CAPÍTULO 19. ALMACENAMIENTO Y PERSISTENCIA DE DATOS 74
19.1. Introducción a localStorage 74
19.2. Lectura de información 74

19.3. Escritura de información 74

19.4. Actualización de registros 75

19.5. Eliminación de registros 75

19.6. Claves de almacenamiento utilizadas 76

19.7. Serialización JSON 76

19.8. Manejo de datos inconsistentes 77

19.9. Copias de seguridad 77

19.10. Limitaciones del almacenamiento local 77

CAPÍTULO 20. SEGURIDAD DEL FRONTEND 78

20.1. Principios de seguridad 78

20.2. Validación de entradas 78

20.3. Protección de rutas 78

20.4. Manejo de credenciales 79

20.5. Riesgos del almacenamiento local 79

20.6. Control de acceso por roles 79

20.7. Protección de información 80

20.8. Recomendaciones de seguridad 80

20.9. Limitaciones de seguridad del frontend 80

CAPÍTULO 21. PRUEBAS Y CONTROL DE CALIDAD 81

21.1. Conceptos de pruebas frontend 81

21.2. Pruebas de componentes 81

21.3. Pruebas de navegación 81

21.4. Pruebas de formularios 81

21.5. Pruebas de autenticación 82

21.6. Pruebas de inventario 82

21.7. Pruebas de reservas 82

21.8. Pruebas de reportes 82

21.9. Pruebas de compatibilidad 82
21.10. Registro de errores 83
21.11. Criterios de aceptación 83
CAPÍTULO 22. COMPILACIÓN Y DESPLIEGUE 84
22.1. Preparación para producción 84
22.2. Ejecución de npm run build 84
22.3. Carpeta dist 84
22.4. Verificación de compilación 84
22.5. Publicación del frontend 85
22.6. Configuración del servidor web 85
22.7. Verificación posterior al despliegue 85
22.8. Actualización de versiones 85
CAPÍTULO 23. MANTENIMIENTO Y SOLUCIÓN DE PROBLEMAS 86
23.1. Mantenimiento preventivo 86
23.2. Mantenimiento correctivo 86
23.3. Actualización de dependencias 86
23.4. Errores frecuentes de React 87
23.5. Errores de rutas 87
23.6. Problemas de almacenamiento 87
23.7. Problemas de estilos 88
23.8. Diagnóstico desde consola 88
23.9. Procedimiento de recuperación 88
23.10. Recomendaciones de continuidad 89
CAPÍTULO 24. ESTÁNDARES Y BUENAS PRÁCTICAS 90
24.1. Convenciones de nomenclatura 90
24.2. Organización del código 90
24.3. Reutilización y modularidad 90

24.4. Accesibilidad 90

24.5. Rendimiento 91

24.6. Calidad del código 91

24.7. Control de versiones 91

24.8. Documentación técnica 91

24.9. Recomendaciones para futuras versiones 92

CAPÍTULO 25. ANEXOS 93

25.1. Glosario técnico ampliado 93

25.2. Inventario de componentes 94

25.3. Inventario de páginas 94

25.4. Matriz de rutas 95

25.5. Matriz de almacenamiento local 96

25.6. Matriz de pruebas 96

25.7. Formato de registro de incidencias 97

25.8. Formato de control de cambios 97

25.9. Referencias técnicas 97

25.10. Historial de versiones del manual 98

CAPÍTULO 1. GENERALIDADES DEL SISTEMA

1.1. Descripción del Centro de Simulación en Salud del SENA

El Centro de Simulación en Salud del Servicio Nacional de Aprendizaje (SENA) es un espacio de formación orientado al fortalecimiento de las competencias de los aprendices del área de la salud. En estos espacios se desarrollan actividades prácticas que permiten representar situaciones relacionadas con la atención de pacientes, el cuidado de la salud y la aplicación de procedimientos clínicos en ambientes controlados.

La simulación permite que los aprendices se familiaricen con equipos, materiales, instrumentos y procedimientos antes de enfrentarse a situaciones reales. Por esta razón, es importante contar con una adecuada organización de los ambientes, los recursos disponibles y los elementos utilizados durante las prácticas.

El sistema de información del Centro de Simulación surge como una herramienta de apoyo para organizar y consultar la información relacionada con estos procesos. Su propósito es facilitar el acceso a los diferentes módulos del centro y ofrecer una interfaz desde la cual los usuarios puedan realizar consultas y gestionar la información disponible.

De esta manera, el proyecto busca contribuir a la organización de los recursos tecnológicos y administrativos del Centro de Simulación, apoyando las actividades que realizan los administradores, instructores y demás usuarios autorizados.

1.2. Descripción del sistema de información

El sistema de información del Centro de Simulación en Salud del SENA es una aplicación web desarrollada con tecnologías frontend modernas. Su interfaz permite acceder a diferentes módulos relacionados con la administración y consulta de información del centro.

La aplicación está construida utilizando React, una biblioteca de JavaScript que permite desarrollar interfaces mediante componentes reutilizables. Para ejecutar y compilar el proyecto se utiliza Vite, mientras que la navegación entre las diferentes páginas se gestiona mediante React Router DOM.

El sistema cuenta con módulos orientados a la gestión de usuarios, ambientes, equipos, inventario, cronogramas, reservas, guías, evaluaciones, simulaciones, reportes y notificaciones.

La interfaz está organizada para que el usuario pueda desplazarse entre las diferentes secciones sin necesidad de ingresar a páginas independientes. Esto facilita la navegación y permite mantener una presentación uniforme en toda la aplicación.

En la versión documentada, parte de la información se almacena mediante localStorage del navegador. Esto significa que los datos permanecen guardados en el navegador donde se utiliza la aplicación, pero no constituyen una base de datos central compartida entre diferentes computadores.

1.3. Antecedentes y necesidad del desarrollo

En los centros de formación que cuentan con ambientes de simulación en salud se maneja información relacionada con equipos, materiales, insumos, usuarios, espacios de práctica y actividades académicas.

Cuando estos datos se administran mediante diferentes archivos o registros independientes, puede resultar difícil consultar la información de manera rápida y mantenerla organizada. También pueden presentarse inconvenientes al identificar la disponibilidad de los recursos, revisar las existencias de los insumos o consultar las actividades programadas.

A partir de esta necesidad surge la propuesta de desarrollar una aplicación que reúna diferentes funcionalidades en una misma interfaz.

El proyecto busca facilitar la consulta y organización de la información, reducir la dispersión de los registros y ofrecer herramientas que apoyen las actividades administrativas y académicas del Centro de Simulación.

Es importante señalar que el sistema constituye una herramienta de apoyo. Su implementación no reemplaza los procedimientos institucionales de control, autorización, mantenimiento o supervisión de los recursos.

1.4. Objetivos del sistema

Objetivo general

Desarrollar una aplicación web frontend para apoyar la gestión y consulta de la información del Centro de Simulación en Salud del SENA, mediante una interfaz organizada, accesible y construida con tecnologías basadas en React.

Objetivos específicos

- Organizar la información correspondiente a los usuarios y sus perfiles.
- Facilitar la consulta de los ambientes y equipos disponibles.
- Proporcionar herramientas para registrar y consultar información del inventario.
- Apoyar la visualización de cronogramas y reservas.
- Integrar secciones destinadas a las guías, prácticas, simulaciones y evaluaciones.
- Facilitar la consulta de reportes y notificaciones.
- Ofrecer una interfaz de navegación clara para los diferentes tipos de usuarios.
- Mantener una estructura de código que facilite futuras modificaciones y mejoras.

1.5. Alcance funcional del frontend

El frontend corresponde a la parte del sistema con la que interactúa directamente el usuario. Su función principal es presentar la información, recibir los datos ingresados mediante formularios, ejecutar las acciones disponibles y mostrar los resultados en pantalla.

Dentro del alcance del proyecto se encuentran la construcción de las páginas, la organización de los componentes, la navegación entre módulos, la presentación de formularios, las validaciones realizadas desde la interfaz y el manejo de información mediante los mecanismos de almacenamiento implementados.

También forma parte del frontend la presentación visual de los estados de los registros, las tablas, los paneles de información y los elementos interactivos.

Este manual se concentra en la arquitectura y el funcionamiento de esa capa de presentación. No debe confundirse el frontend con un sistema completo de servicios backend o con una base de datos institucional centralizada.

1.6. Características principales

El sistema presenta diferentes características que permiten organizar su funcionamiento:

Interfaz modular: las funcionalidades se distribuyen en páginas y componentes.

Navegación interna: permite desplazarse entre las secciones mediante rutas.

Componentes reutilizables: facilita mantener elementos visuales comunes.

Gestión de usuarios: dispone de funcionalidades relacionadas con el acceso y la administración de perfiles.

Gestión de recursos: incluye módulos para consultar y organizar información de equipos, ambientes e inventario.

Persistencia local: utiliza mecanismos del navegador para conservar determinados datos.

Diseño visual: incorpora elementos gráficos, formularios, menús y paneles para facilitar la interacción.

Asistente de consultas: incluye un componente de asistencia cuya capacidad depende de las reglas y datos que tenga implementados.

1.7. Tipos de usuarios

La aplicación contempla diferentes perfiles de usuario, de acuerdo con las funciones que se encuentran implementadas en el sistema.

Entre los perfiles identificados se encuentran:

Administrador: es el perfil destinado a las actividades de administración de usuarios y otras funciones de gestión disponibles en la aplicación.

Instructor: corresponde al usuario relacionado con las actividades de formación y consulta de los recursos del Centro de Simulación.

Usuario invitado: es un perfil con acceso limitado, de acuerdo con las opciones que se encuentren habilitadas.

La disponibilidad real de las funcionalidades depende de las condiciones implementadas en cada módulo y de las restricciones que tenga configuradas la aplicación.

1.8. Roles y permisos

Los roles permiten diferenciar las funciones que pueden realizar los usuarios dentro del sistema. Esta separación ayuda a organizar el acceso a las opciones de administración y consulta.

En el proyecto se identifican los roles de Administrador, Instructor y Usuario invitado. La administración de usuarios contempla estados como Activo e Inactivo, que permiten identificar la condición de cada registro.

La asignación de permisos debe revisarse en el código de las rutas y de los componentes que controlan el acceso. No basta con ocultar una opción del menú para garantizar que una función esté protegida.

En particular, cuando se utiliza únicamente lógica frontend, las restricciones visuales no sustituyen los controles de seguridad que deberían aplicarse en un servidor.

1.9. Restricciones técnicas

El funcionamiento del sistema depende de las tecnologías instaladas, del navegador utilizado y de las características del equipo donde se ejecuta.

Entre las restricciones técnicas que deben tenerse en cuenta se encuentran:

La aplicación necesita un navegador compatible con las tecnologías web utilizadas.

El entorno de desarrollo requiere Node.js y un administrador de paquetes.

La ejecución local depende de la instalación correcta de las dependencias.

Los datos guardados en localStorage pertenecen al navegador y al dispositivo donde se almacenan.

La información local no se sincroniza automáticamente entre diferentes equipos.

Las funciones que requieren servicios externos necesitan una conexión y configuración adicionales.

El comportamiento de los módulos depende de la implementación disponible en la versión del código fuente.

CAPÍTULO 2. FUNDAMENTOS TECNOLÓGICOS

2.1. Introducción al desarrollo frontend

El desarrollo frontend comprende la construcción de la parte visual e interactiva de una aplicación. Es el área encargada de presentar la información y permitir que las personas interactúen con las diferentes funcionalidades del sistema.

En una aplicación web, el frontend se ejecuta principalmente en el navegador. Desde allí se muestran los formularios, menús, botones, tablas, imágenes y demás elementos que conforman la interfaz.

En el proyecto del Centro de Simulación, el frontend permite que los usuarios accedan a los módulos disponibles y realicen las operaciones que se encuentran implementadas.

Para desarrollar esta interfaz se utilizan tecnologías como HTML, CSS y JavaScript, junto con React y las herramientas de configuración del proyecto.

2.2. ¿Qué es React?

React es una biblioteca de JavaScript utilizada para construir interfaces de usuario. Su principal característica es que permite dividir una aplicación en componentes pequeños que pueden organizarse, reutilizarse y mantenerse de manera independiente.

En lugar de construir una página completa con todo su contenido en un solo archivo, React permite distribuir la interfaz en diferentes componentes. Cada uno puede encargarse de una parte específica, como una barra de navegación, un formulario, una tabla o un panel de información.

En el Centro de Simulación, React permite organizar las páginas y funcionalidades del sistema en una estructura modular.

Esta forma de desarrollo facilita la identificación de los archivos responsables de cada sección y permite realizar modificaciones sin tener que reconstruir toda la aplicación.

2.3. Características de React

React ofrece características que resultan útiles para desarrollar aplicaciones con diferentes módulos y elementos interactivos.

Entre sus características principales se encuentran:

Componentes: permiten dividir la interfaz en unidades independientes y reutilizables.

Props: facilitan el envío de información desde un componente principal hacia componentes secundarios.

Estado: permite controlar información que puede cambiar durante el uso de la aplicación.

Hooks: proporcionan funciones para trabajar con estados, efectos y otras características de React.

Renderizado declarativo: permite describir cómo debe mostrarse la interfaz de acuerdo con los datos disponibles.

Reutilización: permite utilizar un mismo componente en diferentes partes del sistema.

2.4. Arquitectura basada en componentes

La arquitectura basada en componentes consiste en dividir una aplicación en unidades funcionales que trabajan de manera coordinada para construir la interfaz.

Por ejemplo, una página del sistema puede estar compuesta por una barra superior, un menú lateral, un área principal y varios elementos de contenido.

Cada uno de estos elementos puede desarrollarse como un componente independiente.

Esta organización permite identificar más fácilmente dónde se encuentra una funcionalidad y evita repetir innecesariamente bloques de código.

En el proyecto del Centro de Simulación se encuentran componentes destinados a la navegación, la protección de rutas, las notificaciones y el asistente virtual, entre otros.

2.5. JavaScript y JSX

JavaScript es el lenguaje de programación utilizado para implementar la lógica de la aplicación. Permite gestionar eventos, validar información, realizar operaciones con datos y controlar el comportamiento de los componentes.

JSX es una extensión de sintaxis utilizada con React que permite escribir estructuras similares a HTML dentro del código JavaScript.

Por ejemplo:

```
function Mensaje() {
  return (
    <div>
      <h2>Centro de Simulación en Salud</h2>
      <p>Bienvenido al sistema.</p>
    </div>
  );
}

export default Mensaje;
```

En este ejemplo se define un componente funcional que devuelve una estructura visual. React interpreta JSX y lo transforma en los elementos necesarios para mostrar el contenido en el navegador.

2.6. HTML5 y CSS3

HTML es el lenguaje utilizado para definir la estructura de los contenidos de una página web. Permite organizar títulos, párrafos, formularios, tablas, imágenes y otros elementos.

CSS se utiliza para establecer la apariencia visual de esos elementos. Mediante sus reglas se pueden definir colores, tamaños, espacios, bordes, alineaciones y distribución del contenido.

En el sistema del Centro de Simulación, HTML y CSS permiten construir una interfaz que facilite la consulta de información y mantenga una presentación coherente entre los diferentes módulos.

2.7. Vite como herramienta de desarrollo

Vite es una herramienta utilizada para configurar y ejecutar proyectos frontend. Proporciona un servidor de desarrollo que permite visualizar los cambios realizados en el código mientras se trabaja en la aplicación.

También permite preparar una versión optimizada del proyecto para su publicación.

En el Centro de Simulación, Vite forma parte del entorno de desarrollo y se utiliza mediante los comandos definidos en el archivo package.json.

Los comandos disponibles deben consultarse directamente en dicho archivo para evitar ejecutar instrucciones que no estén configuradas.

2.8. React Router DOM

React Router DOM es una biblioteca utilizada para administrar la navegación entre las diferentes páginas de una aplicación React.

Mediante las rutas es posible asociar una dirección con un componente específico. De esta manera, cuando el usuario ingresa a una sección, la aplicación muestra el contenido correspondiente.

En el proyecto del Centro de Simulación, React Router DOM permite organizar las direcciones de acceso a los módulos, como el inventario, los ambientes, los equipos, los reportes y las demás páginas disponibles.

La configuración de las rutas se encuentra centralizada principalmente en el archivo App.jsx.

2.9. Bibliotecas y dependencias utilizadas

Las bibliotecas y dependencias son paquetes de código que permiten incorporar funcionalidades sin tener que desarrollar todas las herramientas desde cero.

En el proyecto se utilizan tecnologías y paquetes que apoyan la construcción de la interfaz, la navegación y la generación de documentos.

Entre las dependencias identificadas en el proyecto se encuentran React, React DOM, React Router DOM, Vite, jsPDF y jsPDF-AutoTable.

React y React DOM se utilizan para construir y representar la interfaz. React Router DOM permite gestionar la navegación. Vite facilita el desarrollo y la compilación. Por su parte, jsPDF y jsPDF-AutoTable permiten generar documentos PDF y organizar información en tablas.

Antes de actualizar una dependencia se recomienda revisar su versión, compatibilidad y documentación oficial.

2.10. Ventajas de las tecnologías implementadas

La combinación de React y Vite permite desarrollar una aplicación modular y mantener un entorno de trabajo ágil.

React facilita la organización de las páginas y componentes, mientras que Vite permite ejecutar el proyecto durante el desarrollo y preparar los archivos de producción.

React Router DOM ayuda a estructurar la navegación, y las bibliotecas adicionales permiten incorporar funcionalidades específicas.

En conjunto, estas herramientas ofrecen una base tecnológica que puede mantenerse y ampliarse conforme evolucionen las necesidades del Centro de Simulación.

CAPÍTULO 3. REQUISITOS DEL ENTORNO DE DESARROLLO

3.1. Requisitos de hardware

Recurso	Recomendación
Procesador	Intel Core i3 o equivalente, preferiblemente superior
Memoria RAM	8 GB o más
Almacenamiento	10 GB libres como mínimo para herramientas y proyecto
Pantalla	Resolución de 1366 × 768 o superior
Conexión	Internet para descargar dependencias y consultar documentación

Estos valores son recomendaciones generales y no requisitos oficiales del proyecto. El rendimiento puede variar según el sistema operativo y las herramientas instaladas.

3.2. Requisitos de software

Para desarrollar y mantener la aplicación se requieren herramientas que permitan editar el código, ejecutar los comandos y visualizar los resultados.

Las principales herramientas recomendadas son:

Node.js.

npm, incluido con Node.js.

Visual Studio Code u otro editor compatible.

Git para el control de versiones.

Un navegador web actualizado.

Los archivos del proyecto del Centro de Simulación.

3.3. Sistema operativo compatible

El desarrollo puede realizarse en sistemas operativos compatibles con Node.js, como Windows, macOS y distribuciones Linux.

En el entorno institucional se recomienda mantener una versión actualizada del sistema operativo y contar con permisos suficientes para instalar herramientas y ejecutar los comandos necesarios.

La compatibilidad debe comprobarse antes de realizar el despliegue, especialmente cuando se utilicen rutas de archivos, variables de entorno o configuraciones específicas del sistema operativo.

3.4. Instalación de Node.js

Node.js es un entorno que permite ejecutar JavaScript fuera del navegador. En los proyectos frontend se utiliza principalmente para ejecutar herramientas de desarrollo y administrar dependencias.

Para instalarlo, se debe ingresar a su página oficial y seleccionar una versión compatible con el proyecto.

Después de completar la instalación, se puede verificar desde una terminal mediante los siguientes comandos:

```
node -v
npm -v
```

El primer comando muestra la versión de Node.js y el segundo muestra la versión de npm.

Si ambos comandos devuelven un número de versión, significa que las herramientas se encuentran disponibles desde la terminal.

3.5. Instalación de Visual Studio Code

Visual Studio Code es un editor de código que facilita el desarrollo de aplicaciones mediante herramientas de edición, búsqueda, terminal integrada y extensiones.

Para instalarlo se debe descargar el instalador desde la página oficial y seguir las instrucciones correspondientes al sistema operativo.

Una vez instalado, se recomienda abrir la carpeta principal del proyecto desde la opción Archivo → Abrir carpeta.

Esto permite trabajar con todos los archivos relacionados con la aplicación y acceder a su estructura desde el explorador lateral.

3.6. Instalación de Git

Git es un sistema de control de versiones que permite registrar los cambios realizados en los archivos de un proyecto.

Su utilización facilita el seguimiento de las modificaciones, la recuperación de versiones anteriores y el trabajo coordinado entre diferentes desarrolladores.

Después de instalar Git, se puede verificar su disponibilidad mediante:

git --version

Para utilizarlo en el proyecto se recomienda crear un repositorio y registrar los cambios mediante confirmaciones o commits con mensajes descriptivos.

3.7. Configuración del entorno

Una vez instaladas las herramientas necesarias, se debe preparar el entorno de desarrollo.

El procedimiento consiste en abrir la carpeta del proyecto, identificar el archivo package.json y ejecutar la instalación de las dependencias.

Se recomienda no modificar archivos de configuración sin conocer su función, especialmente aquellos relacionados con Vite, React y las dependencias del proyecto.

También es importante conservar una copia de seguridad antes de realizar cambios importantes.

3.8. Verificación de herramientas

Antes de comenzar a modificar el código se recomienda comprobar que todas las herramientas estén instaladas correctamente.

Para ello se pueden ejecutar los siguientes comandos:

node -v
npm -v
git --version

Después se debe comprobar que el proyecto tenga su archivo package.json y que la carpeta src contenga los archivos fuente.

La verificación permite identificar problemas de instalación antes de iniciar el desarrollo.

3.9. Preparación del proyecto

Para preparar el proyecto se debe disponer de una copia completa de los archivos de la aplicación.

Si el proyecto se encuentra comprimido, primero se debe extraer su contenido en una carpeta de trabajo.

Posteriormente se abre esa carpeta desde Visual Studio Code y se utiliza la terminal integrada para instalar las dependencias.

npm install

Una vez finalizada la instalación, se puede ejecutar el servidor de desarrollo utilizando el comando definido en el archivo package.json.

Antes de continuar, es recomendable comprobar que la aplicación abra correctamente y que no existan errores importantes en la consola del navegador.

CAPÍTULO 4. INSTALACIÓN Y CONFIGURACIÓN DEL PROYECTO

4.1. Estructura inicial del proyecto

La estructura de un proyecto React permite organizar los archivos según su responsabilidad.

En el Centro de Simulación se dispone de una carpeta principal que contiene los archivos de configuración, las dependencias y el código fuente de la aplicación.

Dentro de la carpeta src se encuentran las páginas, los componentes, los contextos, los diseños generales y otros recursos utilizados para construir la interfaz.

Esta organización permite identificar las partes del sistema y facilita el trabajo de mantenimiento.

Captura recomendada: insertar una imagen del explorador de Visual Studio Code mostrando la estructura real de carpetas.

4.2. Archivo package.json

El archivo package.json contiene información importante sobre el proyecto, sus dependencias y los comandos disponibles.

Entre sus elementos principales se encuentran el nombre del proyecto, la versión, los paquetes requeridos y los scripts de ejecución.

En este archivo se definen los comandos que permiten iniciar el servidor de desarrollo, compilar la aplicación y ejecutar otras tareas.

Es importante revisar este archivo antes de instalar paquetes o modificar la configuración del proyecto.

4.3. Dependencias principales

Las dependencias principales son los paquetes que la aplicación necesita para funcionar.

En el proyecto del Centro de Simulación se utilizan React y React DOM para construir la interfaz, además de React Router DOM para administrar las rutas.

También se encuentran bibliotecas orientadas a la generación de documentos, como jsPDF y jsPDF-AutoTable.

Cada dependencia cumple una función específica y debe mantenerse en versiones compatibles con el resto del proyecto.

4.4. Dependencias de desarrollo

Las dependencias de desarrollo son herramientas que se utilizan principalmente durante la construcción, revisión y compilación del proyecto.

En esta categoría se encuentra Vite, que proporciona el servidor de desarrollo y las herramientas de compilación.

También pueden existir herramientas de revisión de código y configuración, según lo establecido en el archivo package.json.

Se recomienda evitar eliminar paquetes sin comprobar primero si son utilizados por los scripts o por la configuración del proyecto.

4.5. Instalación de paquetes

La instalación de paquetes permite descargar las bibliotecas necesarias para ejecutar la aplicación.

El procedimiento habitual consiste en abrir la terminal dentro de la carpeta principal del proyecto y ejecutar:

npm install

Este comando utiliza la información del archivo package.json y, cuando está disponible, el archivo de bloqueo de dependencias para instalar los paquetes correspondientes.

Si durante la instalación aparece un error, se debe revisar el mensaje de la terminal, comprobar la conexión y verificar las versiones de Node.js y npm.

4.6. Configuración de Vite

Vite se encarga de proporcionar el entorno necesario para ejecutar y compilar el proyecto frontend.

Su configuración puede encontrarse en un archivo como vite.config.js, dependiendo de la estructura del proyecto.

En esta configuración se pueden definir opciones relacionadas con los complementos, las rutas base y otros parámetros de compilación.

Cualquier modificación debe realizarse con cuidado y comprobarse mediante una ejecución de desarrollo y una compilación posterior.

4.7. Configuración del punto de entrada

El punto de entrada es el archivo desde el cual se inicia la aplicación.

En el proyecto se utiliza main.jsx para cargar React y representar el componente principal dentro del documento HTML.

Este archivo establece la conexión entre la estructura inicial de la aplicación y el contenido que se presenta en el navegador.

Su modificación puede afectar el inicio de todo el sistema, por lo que se recomienda revisar cuidadosamente cualquier cambio.

4.8. Ejecución del servidor local

El servidor local permite visualizar la aplicación durante el desarrollo.

Para iniciar el proyecto se debe abrir la terminal en la carpeta principal y ejecutar:

npm run dev

Vite mostrará una dirección local que puede abrirse desde el navegador.

Desde esa dirección se puede revisar el funcionamiento de las páginas y comprobar los cambios realizados en el código.

El servidor local está destinado al desarrollo y no debe confundirse con un despliegue institucional de producción.

4.9. Compilación del proyecto

La compilación permite preparar una versión de la aplicación para su publicación.

En el proyecto se utiliza el comando:

npm run build

Cuando el proceso termina correctamente, Vite genera los archivos de producción en la carpeta dist.

Estos archivos contienen los recursos necesarios para servir la interfaz desde un servidor web compatible.

Antes de publicar la compilación, se recomienda revisar los mensajes de la terminal y probar la versión generada.

4.10. Visualización de errores de ejecución

Durante el desarrollo pueden presentarse errores relacionados con la sintaxis, las dependencias, las rutas, los componentes o el manejo de información.

Para identificarlos se deben revisar dos lugares:

La terminal donde se ejecuta Vite.

La consola de desarrollador del navegador.

Los mensajes de error suelen indicar el archivo, la línea o el tipo de problema que se ha presentado.

Es recomendable leer el mensaje completo antes de realizar cambios, ya que modificar archivos sin identificar la causa puede generar errores adicionales.

CAPÍTULO 5. ARQUITECTURA Y ESTRUCTURA DEL FRONTEND

5.1. Descripción general de la arquitectura

La arquitectura del frontend define la manera en que se encuentran organizados los archivos y cómo se relacionan los diferentes elementos que forman parte de la aplicación.

En el proyecto del Centro de Simulación se utiliza una estructura basada en React, en la cual las páginas representan las diferentes secciones del sistema y los componentes permiten reutilizar elementos de la interfaz.

Esta organización facilita el mantenimiento del código, ya que permite identificar dónde se encuentra cada funcionalidad y realizar modificaciones de manera más ordenada.

La arquitectura también permite separar responsabilidades. Por ejemplo, los elementos de navegación se encuentran organizados en componentes, mientras que las páginas contienen la información y las funcionalidades correspondientes a cada módulo.

5.2. Organización de carpetas

La organización de carpetas permite mantener los archivos del proyecto clasificados según su función.

La carpeta principal contiene los archivos de configuración y las dependencias del proyecto. Dentro de ella se encuentra la carpeta src, que almacena el código fuente de la aplicación.

En el proyecto se identifican carpetas destinadas a páginas, componentes, contextos, diseños generales, prácticas, guías y recursos gráficos.

Esta distribución facilita el trabajo de los desarrolladores y permite localizar los archivos que necesitan modificarse cuando se realiza una actualización.

Se recomienda mantener esta organización y evitar guardar archivos sin una ubicación definida.

5.3. Carpeta Components

La carpeta Components contiene elementos reutilizables que forman parte de la interfaz del sistema.

Un componente puede representar una barra de navegación, un menú, una ventana de notificaciones o cualquier otro elemento que se utilice en diferentes secciones.

En esta carpeta se encuentran componentes como Sidebar, Navbar, PrivateRoute, NotificationPanel e IAButton.

Cada componente tiene una responsabilidad determinada. Por ejemplo, Sidebar participa en la navegación lateral, mientras que Navbar representa la barra superior de la interfaz.

Separar estos elementos permite mantener un código más organizado y facilita realizar cambios visuales sin tener que modificar todas las páginas.

5.4. Carpeta Pages

La carpeta Pages reúne los archivos que representan las diferentes páginas del sistema.

Cada página corresponde a una sección funcional a la que el usuario puede acceder mediante la navegación de la aplicación.

Entre las páginas identificadas se encuentran las relacionadas con el inicio de sesión, registro, recuperación de contraseña, dashboard, inventario, equipos, ambientes, cronograma, evaluaciones, guías, reportes, notificaciones y usuarios.

La separación de las páginas permite que cada módulo tenga su propio espacio de desarrollo.

Cuando se necesita modificar una funcionalidad específica, el desarrollador puede localizar el archivo correspondiente y revisar su estructura antes de realizar cambios.

5.5. Carpeta Context

La carpeta Context contiene mecanismos utilizados para compartir información entre diferentes componentes.

En el proyecto se encuentra AuthContext.jsx, relacionado con el manejo del contexto de autenticación.

El uso de Context API permite que determinados datos estén disponibles para varios componentes sin necesidad de transmitirlos manualmente a través de muchos niveles de propiedades.

En este caso, el contexto permite organizar la información relacionada con el usuario y las operaciones de acceso que utiliza la aplicación.

5.6. Carpeta Layouts

La carpeta Layouts contiene estructuras generales que sirven como base visual para diferentes páginas.

En el proyecto se encuentra DashboardLayout.jsx, que organiza elementos comunes del entorno principal de la aplicación.

Este tipo de estructura permite mantener una presentación uniforme entre las diferentes secciones y evita repetir los mismos elementos visuales en cada página.

El uso de un layout facilita modificar la distribución general del sistema desde un punto centralizado.

5.7. Carpeta Practicas

La carpeta Practicas contiene archivos relacionados con las actividades de práctica del Centro de Simulación.

Su función dentro del proyecto debe documentarse revisando los archivos que la componen, sus importaciones y la relación que mantienen con las páginas principales.

En términos generales, esta sección permite organizar los recursos de software que apoyan la representación y consulta de actividades prácticas.

Cuando se incorporen nuevas prácticas, se recomienda conservar una estructura uniforme en los archivos y mantener una relación clara entre los contenidos y los módulos que los utilizan.

5.8. Carpeta Guias

La carpeta Guias contiene recursos relacionados con las guías utilizadas dentro del sistema.

Estos archivos permiten organizar contenidos que pueden estar relacionados con los procesos de formación y las actividades del Centro de Simulación.

Antes de modificar esta carpeta, se debe revisar el tipo de archivos que contiene y la forma en que son llamados desde las páginas de la aplicación.

Se recomienda mantener nombres descriptivos y evitar duplicar archivos que cumplen la misma función.

5.9. Carpeta de recursos e imágenes

Los recursos gráficos permiten complementar la interfaz mediante logotipos, fotografías, ilustraciones e imágenes relacionadas con el Centro de Simulación.

En el proyecto se dispone de recursos visuales asociados con la identidad del SENA, el Centro de Simulación, los ambientes, los equipos y los elementos utilizados en el área de salud.

Estos archivos deben conservarse en las ubicaciones esperadas por los componentes que los utilizan.

Cuando se reemplace una imagen, es necesario comprobar que el nombre y la extensión coincidan con la referencia utilizada en el código.

También se recomienda optimizar el tamaño de las imágenes para evitar que afecten innecesariamente el tiempo de carga.

5.10. Archivo App.jsx

El archivo App.jsx cumple una función importante dentro de la organización general de la aplicación, ya que contiene la configuración principal de las rutas.

En este archivo se relacionan las direcciones de navegación con las páginas correspondientes.

Entre las rutas identificadas se encuentran las destinadas al inicio de sesión, registro, dashboard, ambientes, equipos, inventario, cronograma, evaluaciones, guías, notificaciones, perfil, reportes, ajustes, simulaciones y usuarios.

Su organización permite que React Router DOM determine qué página debe mostrarse según la dirección visitada.

Antes de agregar una ruta nueva, se recomienda revisar las importaciones, la configuración existente y las condiciones de acceso.

5.11. Archivo main.jsx

El archivo main.jsx representa el punto de entrada de la aplicación React.

Desde este archivo se inicializa la aplicación y se establece la representación del componente principal dentro del documento HTML.

También puede incluir configuraciones generales necesarias para que el sistema funcione correctamente.

Por su importancia, se recomienda realizar modificaciones únicamente cuando exista una necesidad técnica clara y después de comprobar que los cambios no afectan la inicialización del proyecto.

5.12. Relación entre componentes

Los componentes trabajan de manera conjunta para construir las diferentes páginas de la aplicación.

Una página puede utilizar componentes de navegación, elementos visuales, formularios y paneles de información.

Por ejemplo, el entorno principal puede integrar el menú lateral, la barra superior y el contenido de la página seleccionada.

Esta relación permite mantener una estructura modular y facilita reutilizar los mismos elementos en diferentes partes del sistema.

Cuando se desarrolla un componente nuevo, se debe definir claramente cuál es su responsabilidad, qué información necesita recibir y qué resultado debe presentar.

5.13. Flujo de navegación

El flujo de navegación describe el recorrido que realiza el usuario desde que ingresa al sistema hasta que accede a los módulos disponibles.

En términos generales, el usuario puede ingresar a una página pública, realizar el proceso de acceso y llegar al entorno principal de la aplicación, desde donde puede seleccionar las diferentes secciones.

La navegación se encuentra organizada mediante React Router DOM y las rutas definidas en App.jsx.

El comportamiento de las rutas privadas depende de los componentes y condiciones de protección que se encuentren implementados.

Captura recomendada: insertar una imagen del menú principal y otra del archivo App.jsx, acompañadas de una explicación del recorrido entre páginas.

CAPÍTULO 6. COMPONENTES DE REACT

6.1. Concepto de componente

Un componente es una unidad independiente que permite construir una parte de la interfaz de usuario.

En React, los componentes ayudan a dividir una aplicación en elementos pequeños que pueden organizarse y reutilizarse.

Por ejemplo, una barra de navegación, un formulario de registro o una tarjeta de información pueden desarrollarse como componentes separados.

En el Centro de Simulación, esta organización permite que las diferentes páginas utilicen elementos comunes y mantengan una presentación uniforme.

6.2. Componentes funcionales

Los componentes funcionales son funciones de JavaScript que devuelven elementos que React puede representar en pantalla.

Son una forma habitual de construir interfaces con React y permiten utilizar Hooks para gestionar estados y efectos.

Un ejemplo sencillo es:

```
function Bienvenida() {
  return (
    <h2>
      Bienvenido al Centro de Simulación en Salud
    </h2>
  );
}

export default Bienvenida;
```

Este componente presenta un mensaje de bienvenida. Puede importarse desde otro archivo y utilizarse dentro de una página.

La recomendación es que cada componente tenga una responsabilidad clara y que su nombre permita identificar fácilmente su función.

6.3. Propiedades (Props)

Las propiedades, conocidas como Props, permiten enviar información desde un componente padre hacia un componente hijo.

Esto facilita que un mismo componente pueda mostrar diferentes contenidos dependiendo de los datos que reciba.

Ejemplo:

```
function Tarjeta({ titulo, descripcion }) {
  return (
    <section>
      <h3>{titulo}</h3>
      <p>{descripcion}</p>
    </section>
  );
}
```

El componente recibe dos propiedades: titulo y descripcion.

De esta manera, se puede utilizar varias veces con información diferente sin tener que crear un componente nuevo para cada caso.

6.4. Estado interno

El estado representa información que puede cambiar durante el funcionamiento de un componente.

Por ejemplo, puede utilizarse para controlar si un menú está abierto, si un formulario muestra un mensaje o qué información se encuentra seleccionada.

En React se utiliza el Hook useState para administrar este tipo de información.

Ejemplo:

```
import { useState } from "react";

function Contador() {
  const [cantidad, setCantidad] = useState(0);

  return (
    <button onClick={() => setCantidad(cantidad + 1)}>
      Cantidad: {cantidad}
    </button>
  );
}
```

En este ejemplo, cada vez que el usuario presiona el botón, el valor aumenta y React actualiza la interfaz.

6.5. Eventos

Los eventos permiten que los componentes respondan a las acciones realizadas por el usuario.

Entre los eventos más utilizados se encuentran los relacionados con los clics, cambios en formularios y envío de información.

Por ejemplo, el evento onClick permite ejecutar una función cuando se presiona un botón.

```
function BotonEjemplo() {
  const mostrarMensaje = () => {
    alert("Acción realizada");
  };

  return (
    <button onClick={mostrarMensaje}>
      Presionar
    </button>
  );
}
```

En el sistema del Centro de Simulación, los eventos permiten ejecutar acciones relacionadas con la navegación, los formularios y la consulta de información.

6.6. Renderizado condicional

El renderizado condicional permite mostrar diferentes elementos dependiendo de una condición.

Por ejemplo, se puede presentar un mensaje cuando no existen registros o mostrar una tabla cuando hay información disponible.

```
function EstadoUsuario({ activo }) {
  return (
    <p>
      {activo ? "Usuario activo" : "Usuario inactivo"}
    </p>
  );
}
```

Esta característica resulta útil para representar los diferentes estados de los registros del sistema.

6.7. Listas y claves

React permite representar colecciones de información mediante funciones como map().

Cuando se muestran listas de elementos, se recomienda proporcionar una propiedad key única y estable para que React pueda identificar cada elemento.

Ejemplo:

```
function ListaUsuarios({ usuarios }) {
  return (
    <ul>
      {usuarios.map((usuario) => (
        <li key={usuario.id}>
          {usuario.nombre}
        </li>
      ))}
    </ul>
  );
}
```

El uso adecuado de las claves ayuda a mantener actualizadas las listas cuando se agregan, modifican o eliminan registros.

6.8. Reutilización de componentes

La reutilización consiste en utilizar un mismo componente en diferentes páginas o secciones de la aplicación.

Esta práctica evita repetir estructuras similares y permite realizar modificaciones desde un único archivo.

Por ejemplo, un botón con un estilo institucional puede utilizarse en formularios de usuarios, inventario y equipos.

Para lograr una reutilización adecuada, se recomienda que los componentes reciban propiedades y no dependan innecesariamente de información específica de una sola página.

6.9. Comunicación entre componentes

La comunicación entre componentes permite compartir información y coordinar las acciones que forman parte de una interfaz.

Una de las formas más comunes consiste en transmitir datos mediante Props desde un componente padre hacia uno hijo.

Cuando varios componentes necesitan acceder a información compartida, también puede utilizarse Context API.

La elección del mecanismo depende de la estructura de la aplicación y de la cantidad de componentes que necesitan utilizar la información.

6.10. Buenas prácticas de desarrollo

Para mantener los componentes organizados se recomienda:

- Utilizar nombres descriptivos.
- Mantener una responsabilidad principal por componente.
- Evitar repetir bloques de código.
- Separar la lógica compleja cuando sea necesario.
- Validar las propiedades que recibe cada componente.
- Utilizar claves estables en las listas.
- Mantener una estructura de archivos uniforme.
- Revisar los errores antes de realizar nuevas modificaciones.

Estas prácticas facilitan la lectura del código y permiten que otros desarrolladores comprendan cómo funciona la aplicación.

CAPÍTULO 7. GESTIÓN DE ESTADOS Y CONTEXTO

7.1. Concepto de estado

El estado es la información que utiliza un componente y que puede cambiar durante la ejecución de la aplicación.

En el frontend, el estado permite controlar datos temporales como formularios, selecciones, mensajes, paneles abiertos y resultados de consultas.

Cuando el estado cambia, React puede actualizar los elementos de la interfaz que dependen de esa información.

Es importante diferenciar el estado de un componente de los datos almacenados de forma persistente, ya que no toda la información se conserva automáticamente después de cerrar o actualizar la página.

7.2. Hook useState

useState es un Hook de React que permite declarar y actualizar el estado dentro de un componente funcional.

Su estructura básica es:

const [valor, setValor] = useState(valorInicial);

El primer elemento representa el valor actual y el segundo es la función que permite actualizarlo.

En el sistema puede utilizarse para administrar información temporal de formularios, ventanas emergentes, filtros y otros elementos interactivos.

Se recomienda actualizar el estado mediante su función correspondiente, en lugar de modificar directamente la variable.

7.3. Hook useEffect

useEffect permite ejecutar determinados efectos secundarios relacionados con un componente.

Puede utilizarse para realizar acciones después de un renderizado, consultar información, registrar eventos o sincronizar determinados datos.

Ejemplo:

```
import { useEffect } from "react";

useEffect(() => {
  console.log("El componente se ha cargado");
}, []);
```

El arreglo de dependencias permite controlar cuándo debe ejecutarse el efecto.

Es importante evitar efectos innecesarios y revisar sus dependencias para prevenir ejecuciones repetidas o comportamientos inesperados.

7.4. Context API

Context API es una funcionalidad de React que permite compartir información entre componentes sin tener que transmitirla manualmente por todos los niveles de la estructura.

Se utiliza cuando varios elementos necesitan consultar un mismo conjunto de datos.

En el Centro de Simulación, el contexto se utiliza para organizar información relacionada con la autenticación.

Esta herramienta permite centralizar determinados datos y facilitar su disponibilidad en las páginas que necesitan utilizarlos.

7.5. AuthContext

El archivo AuthContext.jsx contiene la implementación del contexto relacionado con la autenticación.

Su función es organizar la información del usuario y las operaciones relacionadas con el acceso al sistema.

Entre los elementos que deben revisarse en este archivo se encuentran la creación del contexto, el proveedor, las funciones de acceso y salida, y el mecanismo utilizado para recuperar la información del usuario.

El contexto también se relaciona con el almacenamiento local utilizado por la aplicación.

7.6. Proveedor de autenticación

El proveedor de autenticación es el componente encargado de poner a disposición de sus componentes descendientes la información contenida en el contexto.

En React, normalmente se implementa mediante un componente que utiliza el proveedor del contexto.

La aplicación puede envolver una parte de su estructura dentro de este proveedor para que las páginas y componentes correspondientes puedan acceder a la información de autenticación.

Su ubicación debe revisarse en el código de inicialización de la aplicación.

7.7. Consumo del contexto

El consumo del contexto permite que un componente consulte la información compartida.

En React se puede utilizar el Hook useContext para acceder al valor proporcionado por un contexto.

Ejemplo general:

```
import { useContext } from "react";
import { AuthContext } from "./AuthContext";

function Perfil() {
  const contexto = useContext(AuthContext);

  return (
    <div>
      Información del usuario
    </div>
  );
}
```

7.8. Actualización de información

La actualización de información consiste en modificar los datos que utiliza un componente o que se encuentran disponibles mediante un contexto.

Cuando cambia el estado, React puede volver a representar los elementos que dependen de él.

En el caso de la información del usuario, se debe revisar que las funciones de actualización mantengan la coherencia entre el contexto y los datos almacenados.

Se recomienda evitar que diferentes componentes modifiquen la misma información de manera independiente sin una estrategia clara.

7.9. Manejo de sesiones

El manejo de sesiones permite conservar la información necesaria para identificar al usuario durante el uso de la aplicación.

En el proyecto, el contexto de autenticación utiliza localStorage para conservar información del usuario bajo la clave correspondiente.

Este mecanismo permite recuperar ciertos datos después de actualizar la página. Sin embargo, no representa por sí mismo una sesión segura administrada por un servidor.

Por esta razón, la implementación debe entenderse como un mecanismo de persistencia local y no como una autenticación centralizada con validación de credenciales en backend.

7.10. Consideraciones de persistencia

La persistencia se refiere a la capacidad de conservar información después de determinados eventos, como actualizar o cerrar una página.

En React, el estado de un componente normalmente se pierde cuando se desmonta o se recarga la aplicación, a menos que se utilice algún mecanismo adicional.

En el Centro de Simulación se utiliza almacenamiento local del navegador para conservar determinados datos.

Se recomienda documentar las claves utilizadas, definir procedimientos de respaldo y tener presente que los datos no se comparten automáticamente entre dispositivos.

CAPÍTULO 8. SISTEMA DE RUTAS Y NAVEGACIÓN

8.1. Introducción a React Router DOM

React Router DOM es la biblioteca utilizada para administrar la navegación entre páginas dentro de una aplicación React.

Permite establecer rutas asociadas con componentes específicos y controlar qué contenido debe mostrarse cuando el usuario accede a una dirección determinada.

En el Centro de Simulación, esta herramienta facilita el acceso a los diferentes módulos sin tener que desarrollar una página independiente para cada dirección del navegador.

8.2. Configuración de BrowserRouter

BrowserRouter es un componente de React Router DOM que utiliza el historial del navegador para administrar la navegación.

Permite que la aplicación responda a los cambios de dirección sin tener que recargar completamente el documento en cada desplazamiento.

Su configuración debe revisarse en el punto de entrada de la aplicación o en el archivo donde se haya establecido el sistema de navegación.

8.3. Definición de rutas

En el archivo App.jsx se encuentran las rutas correspondientes a las páginas principales del sistema.

Ruta	Página o funcionalidad
/login	Inicio de sesión
/registro	Registro
/recuperar	Recuperación
/dashboard	Panel principal
/ambientes	Ambientes
/equipos	Equipos
/inventario	Inventario
/cronograma	Cronograma
/evaluaciones	Evaluaciones
/guias	Guías
/reportes	Reportes
/usuarios	Administración de usuarios

La tabla debe completarse con todas las rutas verificadas en el código fuente.

8.4. Navegación entre páginas

La navegación permite que el usuario se desplace entre las diferentes secciones de la aplicación.

Los enlaces y elementos del menú pueden utilizar los mecanismos de React Router DOM para cambiar de página sin realizar una recarga completa.

Una navegación correctamente organizada facilita la utilización del sistema y evita que el usuario tenga que recordar manualmente todas las direcciones.

8.5. Rutas públicas

Las rutas públicas son aquellas que pueden consultarse sin haber iniciado sesión, de acuerdo con la configuración de la aplicación.

En el sistema se identifican páginas relacionadas con el acceso, el registro y la recuperación de información.

La clasificación exacta de cada ruta debe comprobarse en App.jsx y en los componentes utilizados para controlar el acceso.

8.6. Rutas privadas

Las rutas privadas son aquellas cuyo acceso está condicionado por una validación previa.

Su objetivo es evitar que determinados usuarios ingresen directamente a secciones que requieren autenticación.

En el Centro de Simulación se utiliza un componente denominado PrivateRoute para apoyar este control.

Es importante recordar que la protección realizada únicamente en frontend no sustituye la autorización que debe ejecutarse en un servidor cuando se manejan datos sensibles.

8.7. Componente PrivateRoute

PrivateRoute es un componente destinado a controlar el acceso a determinadas rutas.

Su funcionamiento depende de la información de autenticación disponible en el contexto o en el mecanismo que utilice la aplicación.

Cuando las condiciones de acceso se cumplen, permite mostrar el contenido solicitado. De lo contrario, puede redirigir al usuario hacia una página de acceso.

Para realizar cambios en este componente se deben revisar sus condiciones, las rutas que lo utilizan y el comportamiento esperado cuando no existe un usuario autenticado.

8.8. Redirecciones

Las redirecciones permiten enviar al usuario desde una dirección hacia otra.

Pueden utilizarse cuando una persona intenta acceder a una página que requiere autenticación o cuando se necesita dirigirla al panel principal después de completar un proceso.

Es recomendable que las redirecciones sean coherentes con el flujo de navegación y eviten ciclos que impidan acceder al sistema.

8.9. Página no encontrada

Una página no encontrada permite informar al usuario cuando intenta ingresar a una dirección que no se encuentra registrada en la aplicación.

En el proyecto existe una configuración de ruta comodín que redirige hacia la página inicial.

En futuras mejoras puede implementarse una página específica de error 404, con un mensaje claro y un enlace para regresar al inicio.

8.10. Pruebas de navegación

Las pruebas de navegación permiten comprobar que las rutas funcionan correctamente y que cada dirección muestra el contenido esperado.

Se recomienda probar el acceso a las páginas públicas, el ingreso al dashboard, la navegación entre módulos y el comportamiento de las rutas privadas.

También se deben revisar las redirecciones y comprobar que las direcciones inexistentes no produzcan errores inesperados.

Cada prueba debe registrar el resultado obtenido y cualquier novedad identificada.

CAPÍTULO 9. DISEÑO E INTERFAZ DE USUARIO

9.1. Principios de diseño frontend

El diseño frontend debe facilitar la comprensión de la información y permitir que los usuarios realicen sus actividades de manera sencilla.

Para lograrlo, se recomienda mantener una distribución ordenada, utilizar textos claros, organizar los elementos por secciones y conservar una presentación uniforme.

En el Centro de Simulación, la interfaz debe apoyar las actividades de consulta y administración de los recursos, evitando que la información importante quede oculta o resulte difícil de localizar.

9.2. Identidad visual institucional

La identidad visual permite reconocer el sistema y relacionarlo con el Centro de Simulación en Salud del SENA.

Para ello se utilizan recursos gráficos, logotipos y colores que acompañan la presentación de la aplicación.

Los elementos institucionales deben utilizarse de manera proporcionada, evitando distorsionar los logotipos o modificar su apariencia.

Se recomienda mantener los colores, tamaños y ubicaciones de los elementos gráficos de forma coherente en todas las páginas.

9.3. Distribución de la interfaz

La distribución de la interfaz corresponde a la manera en que se organizan los elementos que aparecen en pantalla.

Una estructura puede incluir una barra superior, un menú lateral y un área principal de contenido.

Esta organización permite separar la navegación de la información y facilita que el usuario identifique en qué sección se encuentra.

Los cambios en la distribución deben revisarse en diferentes tamaños de pantalla para evitar problemas de visualización.

9.4. DashboardLayout

DashboardLayout.jsx contiene una estructura general utilizada para organizar el entorno principal de la aplicación.

Este tipo de componente permite reunir elementos comunes de navegación y presentación, evitando que cada página tenga que construir nuevamente toda la estructura.

Su utilización facilita mantener una distribución uniforme entre los módulos y permite realizar modificaciones generales desde un archivo centralizado.

9.5. Barra lateral (Sidebar)

El componente Sidebar representa el menú lateral de navegación.

Su función es presentar accesos a las diferentes secciones del sistema, permitiendo que el usuario se desplace entre los módulos disponibles.

Se recomienda mantener las opciones agrupadas por afinidad funcional y utilizar nombres que permitan comprender fácilmente a dónde conduce cada enlace.

Cuando se agregue una nueva sección, se debe revisar su relación con las rutas definidas en la aplicación.

9.6. Barra superior (Navbar)

El componente Navbar corresponde a la barra superior de la interfaz.

Este elemento puede utilizarse para mostrar información general del sistema, opciones de navegación y accesos relacionados con el usuario.

Su contenido debe conservar una presentación clara y evitar la acumulación de elementos innecesarios.

Los cambios realizados en la barra superior deben probarse en conjunto con el menú lateral y el área principal de contenido.

9.7. Diseño de formularios

Los formularios permiten que los usuarios ingresen información al sistema.

Deben contar con etiquetas comprensibles, campos organizados y mensajes que indiquen cuándo existe un dato incorrecto o incompleto.

En los módulos del Centro de Simulación, los formularios pueden utilizarse para registrar información de usuarios, equipos, ambientes e insumos.

Es importante validar los datos antes de guardarlos y evitar que se acepten valores que puedan generar inconsistencias.

9.8. Tablas y listados

Las tablas y listados permiten presentar información de manera organizada.

En un sistema de gestión, estos elementos facilitan revisar registros, comparar datos y localizar información específica.

Se recomienda utilizar encabezados descriptivos, mantener una distribución uniforme y evitar mostrar demasiada información en una misma fila.

Cuando una tabla permita editar o eliminar registros, las acciones deben estar claramente identificadas.

9.9. Ventanas y paneles emergentes

Las ventanas y paneles emergentes permiten mostrar información adicional sin abandonar completamente la página actual.

Pueden utilizarse para confirmar operaciones, presentar mensajes o mostrar opciones relacionadas con un registro.

Se recomienda que los paneles tengan una estructura clara, botones identificables y una forma sencilla de cerrarse.

También deben probarse con teclado y en diferentes tamaños de pantalla.

9.10. Diseño adaptable

El diseño adaptable permite que la interfaz se ajuste a diferentes tamaños de pantalla.

Esto es importante porque el sistema puede utilizarse desde computadores, tabletas y otros dispositivos con navegadores compatibles.

Se recomienda revisar el comportamiento de los menús, formularios, tablas e imágenes en resoluciones diferentes.

Las tablas extensas pueden requerir desplazamiento horizontal o una presentación alternativa para evitar que el contenido se corte.

9.11. Accesibilidad y usabilidad

La accesibilidad busca que la aplicación pueda ser utilizada por personas con diferentes capacidades y necesidades.

Para mejorarla, se recomienda utilizar etiquetas comprensibles, mantener un contraste adecuado entre texto y fondo, proporcionar nombres accesibles a los controles y permitir la navegación mediante teclado.

La usabilidad se relaciona con la facilidad con la que una persona comprende y utiliza el sistema.

Una interfaz clara, organizada y consistente facilita el aprendizaje y disminuye los errores durante el uso de la aplicación.

CAPÍTULO 10. MÓDULO DE AUTENTICACIÓN Y USUARIOS

10.1. Introducción al módulo

El módulo de autenticación y usuarios permite organizar las funcionalidades relacionadas con el acceso y la administración de las personas que utilizan el sistema.

En el Centro de Simulación, estas funciones son importantes porque permiten diferenciar los perfiles y organizar el acceso a las opciones disponibles.

El módulo contempla páginas de inicio de sesión, registro, recuperación de contraseña, perfil y administración de usuarios.

Su funcionamiento se relaciona con el contexto de autenticación y con los mecanismos de almacenamiento implementados en el frontend.

10.2. Inicio de sesión

El inicio de sesión es el proceso mediante el cual una persona ingresa al sistema utilizando los datos solicitados por el formulario correspondiente.

La aplicación dispone de una página de acceso y de un contexto que organiza la información del usuario.

En el código del proyecto, AuthContext.jsx utiliza la clave usuario de localStorage para conservar información relacionada con el usuario actual.

Es importante comprender que almacenar información en el navegador no equivale a validar credenciales mediante un servicio de autenticación seguro. Para un uso institucional con información sensible se recomienda implementar un backend que valide las credenciales y gestione las sesiones.

10.3. Registro de usuarios

El registro permite incorporar nuevos usuarios a la información que administra la aplicación.

En la implementación identificada, el proceso utiliza la clave usuarios de localStorage para consultar y guardar los registros.

Una vez completado el registro, el flujo continúa con el proceso de acceso definido en el contexto de autenticación.

Se debe revisar que los campos requeridos estén completos, que los datos tengan un formato válido y que no se generen registros duplicados.

También es necesario tener presente que el proyecto contiene otra clave relacionada con la administración de usuarios, por lo que debe verificarse la coherencia entre ambos mecanismos de almacenamiento.

10.4. Recuperación de contraseña

La recuperación de contraseña es una funcionalidad destinada a ayudar al usuario cuando no puede acceder a su cuenta.

La aplicación dispone de una página de recuperación. Su procedimiento debe documentarse de acuerdo con las operaciones que realmente ejecuta el código.

No se debe afirmar que el sistema envía correos electrónicos, genera enlaces de recuperación o valida identidades mediante servicios externos si esas funciones no están implementadas.

Para una versión institucional se recomienda que la recuperación se gestione desde un servicio backend con mecanismos seguros de verificación.

10.5. Gestión del perfil

La gestión del perfil permite consultar o modificar información relacionada con el usuario que utiliza la aplicación.

En el proyecto se identifica una página denominada Miperfil, relacionada con esta funcionalidad.

Su propósito es reunir la información personal disponible y facilitar las operaciones que se encuentren habilitadas.

Las modificaciones deben validar los datos ingresados y conservar la coherencia con la información utilizada por el contexto de autenticación.

10.6. Administración de usuarios

La administración de usuarios permite consultar y gestionar los registros disponibles en el sistema.

En la página Usuarios.jsx se identifican funcionalidades relacionadas con la búsqueda por nombre, correo electrónico y rol.

También se contemplan los estados Activo e Inactivo, que permiten diferenciar la condición de los registros.

La página utiliza la clave usuariosCentroSimulacion para consultar y almacenar información de usuarios.

Esta clave es diferente de la clave usuarios utilizada por el proceso de registro, por lo que se debe revisar si ambas estructuras se encuentran sincronizadas antes de realizar modificaciones.

10.7. Roles del sistema

Los roles permiten clasificar a los usuarios según las responsabilidades que tienen dentro de la aplicación.

En el proyecto se identifican los siguientes perfiles:

Administrador.

Instructor.

Aprendiz

Usuario invitado.

Cada perfil debe relacionarse con las opciones que tiene habilitadas.

Es recomendable documentar los permisos reales mediante una matriz que indique qué módulos puede consultar o administrar cada rol.

No se deben atribuir permisos que no estén definidos en el código.

10.8. Estados activo e inactivo

Los estados permiten identificar la condición de un registro de usuario.

El estado Activo representa un registro habilitado según la lógica de la aplicación, mientras que Inactivo permite identificar un registro que se encuentra deshabilitado.

La administración de estos estados facilita organizar la información y consultar la situación de los usuarios.

Debe comprobarse en el código si cambiar el estado también bloquea el inicio de sesión o si únicamente modifica la clasificación visual del registro.

10.9. Validación de formularios

La validación de formularios permite comprobar que la información ingresada cumpla con las condiciones necesarias antes de procesarla.

Entre las validaciones recomendadas se encuentran la obligatoriedad de determinados campos, la estructura del correo electrónico y la consistencia de los datos.

También se debe revisar que los mensajes de error sean comprensibles y que permitan al usuario corregir la información.

La validación realizada en el navegador mejora la experiencia de uso, pero no reemplaza las validaciones que deberían realizarse en el servidor.

10.10. Almacenamiento de información de usuarios

El almacenamiento de usuarios permite conservar los registros utilizados por los diferentes procesos del módulo.

En la implementación identificada se utilizan, al menos, dos claves de almacenamiento relacionadas con los usuarios: usuarios y usuariosCentroSimulacion.

Esta diferencia debe tenerse presente al realizar modificaciones, ya que una operación puede actualizar una estructura sin modificar automáticamente la otra.

Se recomienda documentar los campos utilizados, establecer un procedimiento de respaldo y evaluar la migración hacia una base de datos centralizada si el sistema va a utilizarse desde varios equipos.

CAPÍTULO 11. MÓDULO DE AMBIENTES

11.1. Descripción funcional

El módulo de ambientes está orientado a organizar la información de los espacios disponibles en el Centro de Simulación en Salud del SENA.

Estos ambientes son importantes para el desarrollo de las actividades prácticas, ya que permiten disponer de espacios adecuados para realizar ejercicios de simulación, demostraciones y procesos de formación.

Desde el frontend, el módulo proporciona una interfaz para consultar la información de los ambientes y acceder a las operaciones que se encuentren implementadas.

11.2. Estructura del módulo

La estructura del módulo está conformada por la página encargada de presentar la información y los elementos visuales que permiten interactuar con ella.

La organización del código debe facilitar la identificación de los formularios, listados, botones y demás componentes relacionados con la gestión de los ambientes.

Cuando se realicen modificaciones, es importante conservar la coherencia entre la información presentada y los datos utilizados por la aplicación.

11.3. Visualización de ambientes

La visualización permite consultar los ambientes registrados en el sistema.

La información puede organizarse mediante tarjetas, listados o tablas, dependiendo del diseño implementado.

Se recomienda que cada ambiente presente información suficiente para identificarlo, como su nombre, descripción, ubicación o estado, siempre que esos campos estén contemplados en el proyecto.

Una presentación ordenada facilita que instructores y administradores localicen el espacio que necesitan consultar.

11.4. Registro y consulta

El registro permite incorporar información de un ambiente, mientras que la consulta facilita revisar los datos existentes.

Los formularios deben solicitar únicamente la información necesaria y presentar mensajes cuando algún campo obligatorio no se encuentre diligenciado.

Antes de guardar un registro se recomienda comprobar que los datos estén completos y que no existan duplicados.

11.5. Reservas de ambientes

Las reservas permiten organizar la utilización de los espacios durante determinadas actividades.

En el proyecto se identifica la clave reservasAmbiente dentro del almacenamiento local.

Este mecanismo permite conservar información de reservas en el navegador, de acuerdo con la lógica implementada.

Se debe comprobar si el sistema valida los cruces de horarios, la disponibilidad y los posibles conflictos entre reservas antes de describir esas funciones como características disponibles.

11.6. Validación de disponibilidad

La validación de disponibilidad busca evitar que se asignen espacios que no puedan utilizarse en una fecha u horario determinado.

Para ello, una implementación puede comparar las fechas, horas y ambientes seleccionados con las reservas existentes.

Es importante que los datos de fecha y hora se manejen de forma consistente para evitar errores de comparación.

La validación debe probarse con diferentes escenarios, incluyendo reservas coincidentes y horarios disponibles.

11.7. Almacenamiento de reservas

La información relacionada con las reservas puede conservarse mediante la clave reservasAmbiente de localStorage.

Al tratarse de almacenamiento local, los datos quedan asociados al navegador donde se ejecuta la aplicación.

Esto significa que una reserva registrada en un equipo no necesariamente estará disponible en otro dispositivo.

Para lograr una administración compartida se recomienda utilizar un servicio backend y una base de datos centralizada.

11.8. Manejo de novedades

Las novedades permiten registrar situaciones que puedan afectar la utilización de un ambiente.

Entre los ejemplos que podrían contemplarse se encuentran trabajos de mantenimiento, daños, restricciones temporales o cambios en la disponibilidad.

Estas situaciones deben documentarse de acuerdo con las funcionalidades que realmente ofrezca el módulo.

Se recomienda utilizar descripciones claras y conservar los registros necesarios para facilitar el seguimiento.

CAPÍTULO 12. MÓDULO DE EQUIPOS

12.1. Descripción funcional

El módulo de equipos está relacionado con la consulta y organización de los elementos utilizados en los procesos de formación y simulación.

En el Centro de Simulación se pueden utilizar diferentes equipos destinados a representar situaciones clínicas y apoyar el desarrollo de las competencias de los aprendices.

La aplicación busca facilitar la consulta de la información disponible y apoyar la organización de los recursos tecnológicos.

12.2. Registro de equipos

El registro de equipos permite incorporar información relacionada con los elementos disponibles.

Al diseñar un formulario de registro se deben considerar los campos necesarios para identificar cada equipo y diferenciarlo de los demás.

Entre los datos que pueden resultar útiles se encuentran el nombre, código, descripción, ubicación y estado, siempre que formen parte de la estructura implementada.

Se recomienda validar la información antes de guardarla para evitar registros incompletos o duplicados.

12.3. Consulta de equipos

La consulta permite localizar y revisar los registros existentes.

Una presentación adecuada debe facilitar la identificación del equipo y permitir comprender su condición actual.

Cuando existen numerosos registros, resulta conveniente utilizar filtros o mecanismos de búsqueda que permitan encontrar información por nombre, código o categoría.

12.4. Estados de los equipos

Los estados permiten identificar las condiciones en las que se encuentran los equipos.

En una gestión de recursos pueden utilizarse clasificaciones como disponible, en mantenimiento o fuera de servicio, siempre que el sistema las tenga implementadas.

El significado de cada estado debe ser claro y mantener una representación uniforme en las diferentes pantallas.

12.5. Control de disponibilidad

El control de disponibilidad permite identificar si un equipo puede utilizarse para una actividad determinada.

Para ello, se deben considerar los datos que maneje el sistema y las condiciones que determinen su utilización.

Si existen registros de mantenimiento o uso, estos pueden ayudar a identificar situaciones que afecten la disponibilidad.

Las reglas deben estar claramente definidas para evitar que un equipo aparezca como disponible cuando realmente no puede utilizarse.

12.6. Registro de uso

El registro de uso permite documentar cuándo se utiliza un equipo y en qué actividad participa.

En el proyecto se identifica la clave usosEquipos dentro del almacenamiento local.

La información guardada mediante este mecanismo puede utilizarse para conservar registros de utilización, según la lógica implementada.

Se recomienda revisar los campos almacenados y comprobar que las operaciones de consulta y actualización mantengan la información consistente.

12.7. Gestión de novedades

La gestión de novedades permite documentar situaciones relacionadas con el funcionamiento o las condiciones de los equipos.

Por ejemplo, se pueden registrar observaciones sobre daños, fallas, mantenimiento o cambios en su ubicación, cuando el módulo contemple estos datos.

Las novedades deben describirse de manera clara y permitir identificar el equipo relacionado.

12.8. Validación de información

La validación permite comprobar que los datos registrados sean correctos y suficientes.

Se recomienda revisar los campos obligatorios, los formatos de identificación y la consistencia de los estados.

También se deben probar las operaciones de consulta y actualización para comprobar que los cambios se reflejen correctamente en la interfaz.

CAPÍTULO 13. MÓDULO DE INVENTARIO

13.1. Descripción general

El módulo de inventario permite organizar la información de los insumos utilizados en el Centro de Simulación en Salud del SENA.

Su propósito es facilitar el registro, consulta y seguimiento de los elementos disponibles para las actividades de formación y práctica.

Este módulo resulta importante porque permite consultar las cantidades existentes y reconocer situaciones que requieren atención, como niveles bajos de existencias o fechas de vencimiento próximas.

13.2. Estructura del inventario

La página Inventario.jsx contiene la implementación relacionada con la gestión de los productos e insumos.

En ella se encuentran operaciones destinadas a consultar y actualizar la información del inventario.

El código utiliza almacenamiento local para conservar los registros y los movimientos relacionados con los productos.

Esta estructura permite organizar la información dentro de la aplicación, aunque no constituye una base de datos centralizada.

13.3. Registro de insumos

El registro de insumos permite incorporar nuevos elementos al inventario.

La información debe permitir identificar cada producto y conocer sus características principales.

Entre los campos identificados en el módulo se encuentran datos relacionados con la cantidad, el stock mínimo, la fecha de vencimiento y la ubicación.

Antes de guardar un producto se recomienda revisar que la información sea válida y que los valores numéricos correspondan con las cantidades registradas.

13.4. Consulta y búsqueda

La consulta permite revisar los productos registrados en el inventario.

La búsqueda facilita localizar un insumo sin tener que recorrer todos los registros manualmente.

Se recomienda que los resultados se presenten de forma clara y que permitan identificar rápidamente el nombre del producto, su cantidad y su ubicación.

Cuando el inventario aumenta, los filtros y criterios de búsqueda se convierten en herramientas importantes para mejorar la experiencia de uso.

13.5. Actualización de existencias

La actualización de existencias permite modificar las cantidades registradas cuando se presentan entradas, salidas u otros movimientos.

Es importante que cada actualización se realice de manera controlada y que los valores resultantes sean coherentes.

Por ejemplo, una salida de inventario no debería producir cantidades negativas si las reglas del sistema no lo permiten.

También se recomienda conservar la información de los movimientos para facilitar la revisión de los cambios realizados.

13.6. Control de stock mínimo

El stock mínimo representa la cantidad establecida como referencia para identificar cuándo un producto necesita reposición.

En la implementación identificada se utiliza un valor predeterminado de 20 unidades cuando no se proporciona otro valor para este campo.

El sistema clasifica como bajo stock aquellos productos cuya cantidad es menor o igual al stock mínimo establecido.

Esta condición permite identificar insumos que requieren revisión y facilita la planificación de las reposiciones.

13.7. Control de fechas de vencimiento

El control de fechas de vencimiento permite identificar los productos que tienen una fecha límite de utilización.

Esta información es especialmente importante en los insumos utilizados durante las actividades de simulación en salud.

Se recomienda ingresar las fechas en un formato uniforme y verificar que el sistema interprete correctamente los valores.

También es importante diferenciar entre un producto vencido y uno próximo a vencer, ya que representan situaciones distintas.

13.8. Ubicación de insumos

La ubicación permite identificar el lugar donde se encuentra almacenado un producto.

En el módulo se contempla el campo ubicacion y puede utilizarse el valor predeterminado «Sin ubicación» cuando no se registra un lugar específico.

Mantener actualizada esta información facilita la localización de los insumos y mejora la organización del inventario físico.

Se recomienda utilizar nombres consistentes para las estanterías, gabinetes, depósitos y demás espacios de almacenamiento.

13.9. Movimientos de inventario

Los movimientos permiten registrar cambios relacionados con las cantidades de los productos.

En el proyecto se identifica la clave movimientosInventario para almacenar información relacionada con estas operaciones.

La conservación de los movimientos facilita consultar las modificaciones realizadas y comprender por qué cambió la cantidad de un producto.

Se recomienda que cada movimiento tenga información suficiente para identificar el producto, el tipo de operación, la cantidad y la fecha, según los campos implementados.

13.10. Estados de disponibilidad

Los estados de disponibilidad permiten representar las condiciones de los productos.

Por ejemplo, la clasificación de bajo stock ayuda a reconocer los elementos que necesitan reposición.

La representación visual de estos estados debe ser clara y mantener una relación directa con los valores almacenados.

Se recomienda evitar que los colores sean el único medio para identificar una condición, ya que también debe existir un texto que describa el estado.

13.11. Validaciones

Las validaciones permiten reducir los errores durante el registro y actualización de los productos.

Se recomienda verificar que las cantidades sean numéricas, que las fechas tengan un formato válido y que los campos obligatorios se encuentren completos.

También debe comprobarse que las salidas de inventario no generen valores inconsistentes.

Estas validaciones deben probarse antes de publicar cualquier modificación del módulo.

13.12. Persistencia de datos

El módulo utiliza localStorage para conservar información del inventario y sus movimientos.

Entre las claves identificadas se encuentran inventarioProductos y movimientosInventario.

La persistencia local permite recuperar la información desde el mismo navegador, pero no garantiza que los registros estén disponibles en otros computadores.

Por esta razón, se recomienda realizar respaldos y evaluar una futura integración con una base de datos centralizada si el sistema va a ser utilizado por diferentes usuarios desde varios dispositivos.

CAPÍTULO 14. MÓDULO DE CRONOGRAMA Y RESERVAS

14.1. Descripción del módulo

El módulo de cronograma y reservas está orientado a organizar las actividades que se desarrollan en los espacios de simulación.

Su propósito es facilitar la consulta de las actividades programadas y apoyar la coordinación de los espacios utilizados durante los procesos de formación.

La información presentada debe permitir que los usuarios comprendan las fechas, los horarios y las actividades disponibles.

14.2. Organización de actividades

La organización de actividades permite agrupar la información de acuerdo con los datos registrados en el sistema.

Cada actividad debe contar con información suficiente para identificar su propósito y relacionarla con el ambiente correspondiente, cuando aplique.

Una estructura organizada facilita la consulta y disminuye la posibilidad de confusiones durante la programación.

14.3. Visualización del cronograma

La visualización permite consultar las actividades mediante una presentación ordenada.

Dependiendo de la implementación, la información puede organizarse en listados, tablas o vistas de calendario.

Se recomienda mantener visibles los datos más importantes y utilizar formatos uniformes para representar fechas y horarios.

14.4. Registro de actividades

El registro permite incorporar nuevas actividades a la programación.

Los formularios deben solicitar los datos necesarios y validar que la información esté completa.

También se recomienda revisar la coherencia entre las fechas de inicio y finalización y comprobar que los horarios correspondan con la actividad registrada.

14.5. Reservas de ambientes

Las reservas permiten organizar la utilización de los espacios disponibles.

Esta funcionalidad debe mantener relación con la información de los ambientes y los horarios registrados.

El sistema debe presentar los datos necesarios para identificar la reserva y permitir consultar su estado.

Cuando exista una integración entre cronograma y reservas, se debe comprobar que las modificaciones se reflejen correctamente en ambas secciones.

14.6. Control de fechas y horarios

El control de fechas y horarios permite comprobar que las actividades se encuentren dentro de los períodos establecidos.

Se recomienda utilizar formatos consistentes y validar que no existan fechas incorrectas.

Cuando se gestionan reservas, también es importante revisar los posibles cruces de horarios y la disponibilidad del ambiente seleccionado.

14.7. Validación de información

La validación permite comprobar que las actividades se registren con información suficiente y coherente.

Se deben revisar los campos obligatorios, las fechas, los horarios y las relaciones con los ambientes.

Las pruebas deben incluir registros válidos, información incompleta y posibles conflictos de programación.

CAPÍTULO 15. MÓDULO DE SIMULACIONES Y PRÁCTICAS

15.1. Descripción del módulo

El módulo de simulaciones y prácticas está relacionado con las actividades formativas que se desarrollan en el Centro de Simulación.

Estas actividades permiten representar situaciones relacionadas con la atención en salud y fortalecer los conocimientos adquiridos durante la formación.

La aplicación incorpora una sección destinada a organizar y consultar información relacionada con las simulaciones y prácticas.

15.2. Organización de simulaciones

La organización permite clasificar las simulaciones de acuerdo con la información disponible en el sistema.

Se recomienda utilizar nombres y descripciones que permitan identificar claramente el propósito de cada actividad.

Cuando existan diferentes tipos de simulación, se pueden organizar mediante categorías o secciones, siempre que la implementación lo permita.

15.3. Gestión de prácticas

La gestión de prácticas permite organizar la información relacionada con los ejercicios que realizan los aprendices.

Cada práctica puede estar asociada con una temática, una actividad o determinados recursos del Centro de Simulación.

La aplicación debe presentar los contenidos de forma clara para facilitar su consulta.

15.4. Recursos asociados

Los recursos asociados son los elementos que pueden utilizarse durante una actividad práctica.

Pueden incluir ambientes, equipos, insumos y materiales de apoyo.

La relación entre estos recursos debe documentarse de acuerdo con las funcionalidades reales del proyecto.

Una adecuada identificación de los recursos facilita la preparación de las actividades.

15.5. Navegación de contenidos

La navegación de contenidos permite desplazarse entre las diferentes secciones relacionadas con las prácticas y simulaciones.

Los enlaces y botones deben utilizar nombres comprensibles y permitir identificar el contenido que se abrirá.

También es importante mantener una estructura uniforme para facilitar el uso del módulo.

15.6. Registro de información

El registro permite incorporar o actualizar información relacionada con las actividades, de acuerdo con las operaciones disponibles.

Los datos deben presentarse de forma organizada y utilizar validaciones para evitar registros incompletos.

Antes de realizar cambios se recomienda revisar la estructura de los archivos y los mecanismos de almacenamiento que utiliza el módulo.

15.7. Relación con los ambientes y equipos

Las simulaciones pueden necesitar espacios y equipos específicos para su desarrollo.

Por esta razón, es importante mantener una relación coherente entre la actividad programada y los recursos que requiere.

Cuando el sistema no realice automáticamente esta asociación, se debe dejar claro que la coordinación corresponde al procedimiento administrativo del Centro de Simulación.

CAPÍTULO 16. MÓDULO DE GUÍAS Y EVALUACIONES

16.1. Gestión de guías

El módulo de guías está destinado a organizar y presentar los recursos de apoyo utilizados en los procesos de formación.

Las guías permiten orientar las actividades y proporcionar información para el desarrollo de las prácticas.

La aplicación dispone de una sección destinada a su consulta, cuya estructura debe mantenerse organizada y relacionada con los contenidos correspondientes.

16.2. Organización de contenidos

La organización de contenidos permite agrupar la información para facilitar su consulta.

Se recomienda utilizar títulos descriptivos y mantener una estructura uniforme en los archivos y recursos asociados.

La clasificación debe facilitar que los usuarios identifiquen rápidamente el material que necesitan.

16.3. Visualización de guías

La visualización permite acceder a los contenidos disponibles desde la interfaz.

Se debe comprobar que los recursos se carguen correctamente y que su presentación sea legible.

Cuando se utilicen documentos o imágenes, se recomienda verificar sus rutas y formatos antes de publicar cambios.

16.4. Gestión de evaluaciones

El módulo de evaluaciones permite organizar las funcionalidades relacionadas con las actividades de valoración del aprendizaje que se encuentren implementadas.

La presentación de las evaluaciones debe ser clara y permitir comprender las instrucciones y los elementos que debe completar el usuario.

Los cambios en esta sección deben probarse con diferentes escenarios para verificar que los datos se procesen correctamente.

16.5. Registro de resultados

El registro de resultados permite conservar información obtenida durante una evaluación, siempre que la funcionalidad esté implementada.

Se recomienda verificar qué datos se almacenan, dónde se guardan y cómo se relacionan con el usuario o la actividad correspondiente.

No se debe asumir que los resultados se encuentran sincronizados con un sistema académico externo si no existe una integración comprobada.

16.6. Validación de respuestas

La validación permite comprobar que las respuestas ingresadas cumplan las condiciones definidas para la evaluación.

Es importante que los mensajes sean claros y que el usuario pueda reconocer si existe una respuesta incompleta o incorrecta.

También se recomienda probar los diferentes tipos de preguntas y las condiciones de finalización.

16.7. Presentación de resultados

La presentación de resultados permite mostrar la información obtenida al finalizar una evaluación.

Su diseño debe facilitar la lectura y evitar interpretaciones confusas.

Cuando se presenten calificaciones o indicadores, es necesario explicar su significado y comprobar que los cálculos correspondan con las reglas implementadas.

CAPÍTULO 17. MÓDULOS DE REPORTES Y NOTIFICACIONES

17.1. Descripción de reportes

Los reportes permiten presentar información organizada para facilitar su consulta y análisis.

En el Centro de Simulación pueden resultar útiles para revisar datos relacionados con usuarios, inventario, equipos y otras áreas de gestión.

La información que aparece en cada reporte depende de los registros y funcionalidades disponibles en la aplicación.

17.2. Generación de reportes

La generación de reportes consiste en organizar los datos y presentarlos en un formato que facilite su revisión.

Antes de generar un reporte se recomienda verificar que la información esté actualizada y que los filtros utilizados correspondan con el resultado esperado.

Los reportes deben incluir títulos y encabezados comprensibles para facilitar su interpretación.

17.3. Exportación de información

La exportación permite obtener información del sistema en un archivo que pueda consultarse fuera de la aplicación.

Antes de exportar se debe comprobar que los datos estén organizados y que no se incluyan registros innecesarios.

También se recomienda proteger la información personal y evitar compartir archivos que contengan datos que no deban divulgarse.

17.4. Generación de documentos PDF

El proyecto utiliza las bibliotecas jsPDF y jsPDF-AutoTable, que permiten generar documentos PDF desde JavaScript.

jsPDF proporciona funciones para construir documentos, mientras que jsPDF-AutoTable facilita organizar información en tablas.

Estas herramientas pueden utilizarse para generar reportes con encabezados, filas y otros elementos de presentación.

Se recomienda probar los documentos generados para verificar que las tablas no se corten y que el contenido sea legible.

17.5. Notificaciones del sistema

Las notificaciones permiten informar al usuario sobre determinados eventos o situaciones.

En el proyecto se identifica la clave notificacionesSENA dentro del almacenamiento local.

La información guardada mediante este mecanismo puede utilizarse para presentar mensajes en la interfaz.

Se debe revisar qué eventos generan notificaciones y cuáles son las condiciones que permiten marcarlas como leídas o pendientes.

17.6. Panel de notificaciones

El componente NotificationPanel está relacionado con la presentación de las notificaciones.

Su función es organizar los mensajes y permitir que el usuario consulte la información disponible.

Se recomienda mantener una presentación clara, diferenciar los estados de los mensajes y evitar que las notificaciones importantes pasen desapercibidas.

17.7. Preferencias de notificación

Las preferencias permiten conservar determinadas opciones relacionadas con la presentación o comportamiento de las notificaciones.

En el proyecto se identifica la clave preferenciasAjustesSENA dentro del almacenamiento local.

Antes de modificar estas opciones se recomienda revisar cómo se consultan y actualizan desde los componentes correspondientes.

También debe comprobarse que los cambios se mantengan después de actualizar la página.

CAPÍTULO 18. ASISTENTE VIRTUAL DEL SISTEMA

18.1. Descripción del asistente

El sistema incorpora un componente de asistencia que permite interactuar con una interfaz destinada a recibir consultas de los usuarios.

Su propósito es facilitar el acceso a información y proporcionar una experiencia de consulta dentro de la aplicación.

El alcance de las respuestas depende de la lógica programada en el componente y de los datos a los que tenga acceso.

Es importante diferenciar un asistente basado en reglas predefinidas de uno conectado a un servicio de inteligencia artificial. La presencia de un botón o una ventana de chat no significa necesariamente que exista una conexión con un modelo de inteligencia artificial externo.

18.2. Componente IAButton

IAButton es el componente encargado de presentar la interfaz del asistente.

Dentro del proyecto se utiliza como un elemento independiente que puede integrarse en la estructura general de la aplicación.

Su función incluye presentar el acceso al asistente y permitir que el usuario interactúe con las opciones disponibles.

Para modificarlo se deben revisar su estructura JSX, los estados utilizados y las funciones que procesan los mensajes.

18.3. Estructura del componente

La estructura del componente puede incluir elementos visuales como un botón de apertura, un panel de conversación, un campo de entrada y un área donde se muestran los mensajes.

También puede utilizar estados internos para controlar si el panel se encuentra abierto y conservar temporalmente los mensajes de la conversación.

Se recomienda mantener separada la presentación visual de la lógica encargada de interpretar las consultas.

18.4. Interfaz de interacción

La interfaz de interacción debe permitir que el usuario escriba una consulta y visualice la respuesta correspondiente.

Los controles deben contar con etiquetas claras y permitir realizar las acciones de manera sencilla.

También es importante comprobar el comportamiento del panel cuando se abre, se cierra y se reciben mensajes extensos.

18.5. Procesamiento de consultas

El procesamiento de consultas depende de la lógica programada dentro del asistente.

Cuando se utilizan reglas basadas en palabras clave, el componente identifica determinados términos y selecciona una respuesta previamente definida.

Este tipo de funcionamiento puede servir para responder preguntas frecuentes, pero tiene limitaciones cuando el usuario formula preguntas diferentes a las previstas.

Si se desea implementar un asistente con consultas dinámicas, será necesario definir cómo accederá a la información de los módulos y qué mecanismos se utilizarán para procesar las preguntas.

18.6. Consultas sobre usuarios

Las consultas relacionadas con usuarios pueden orientarse a identificar registros, roles y estados.

Para ofrecer respuestas correctas, el asistente necesita acceder a la misma información que utiliza el módulo de administración de usuarios.

En el proyecto se identifican diferentes claves de almacenamiento relacionadas con usuarios, por lo que se debe revisar cuál contiene los registros utilizados en cada operación.

No es recomendable que el asistente responda con cantidades o nombres fijos si se espera que consulte datos actualizados.

18.7. Consultas sobre inventario

Las consultas sobre inventario pueden estar relacionadas con las cantidades disponibles, los productos con bajo stock, las fechas de vencimiento y la ubicación de los insumos.

Para que estas respuestas sean confiables, el asistente debe consultar la información almacenada por el módulo de inventario.

La respuesta debe basarse en los registros reales y no en ejemplos predefinidos.

También se recomienda que el asistente indique cuando no encuentra información suficiente para responder una consulta.

18.8. Consultas sobre equipos y ambientes

Las consultas sobre equipos y ambientes pueden facilitar la localización de recursos y la revisión de su disponibilidad.

Para responder correctamente, el asistente necesita consultar los datos utilizados por los módulos correspondientes.

Si la información no está centralizada o no se encuentra actualizada, las respuestas pueden presentar diferencias frente a los registros reales.

Por esta razón, es necesario establecer una fuente de información única antes de implementar consultas dinámicas.

18.9. Limitaciones del asistente

El asistente puede presentar limitaciones relacionadas con la interpretación de preguntas, la disponibilidad de datos y el alcance de las reglas programadas.

Si funciona mediante palabras clave, puede no comprender preguntas formuladas de manera diferente a las previstas.

Si utiliza información almacenada localmente, sus respuestas dependerán de los registros disponibles en ese navegador.

También debe tenerse presente que no debe utilizarse como sustituto de los procedimientos institucionales ni como fuente de decisiones clínicas.

18.10. Mejoras futuras

Entre las posibles mejoras se encuentran la integración con un servicio de inteligencia artificial, la consulta dinámica de los módulos y la conexión con una fuente de datos centralizada.

También sería conveniente incorporar controles de acceso, registro de consultas, mensajes de advertencia y mecanismos para evitar que se exponga información personal.

Cualquier integración futura deberá evaluarse desde los aspectos técnicos, de seguridad y de protección de datos.

CAPÍTULO 19. ALMACENAMIENTO Y PERSISTENCIA DE DATOS

19.1. Introducción a localStorage

localStorage es una funcionalidad del navegador que permite almacenar información en forma de pares clave-valor.

En el proyecto del Centro de Simulación se utiliza para conservar determinados registros relacionados con usuarios, inventario, notificaciones, reservas y preferencias.

Una de sus características es que los datos permanecen disponibles después de cerrar o actualizar la página, siempre que no se eliminen desde el navegador o mediante el código.

Sin embargo, la información se encuentra asociada al navegador y al dispositivo donde se guarda. Por esta razón, no debe considerarse una base de datos institucional compartida.

19.2. Lectura de información

La lectura permite recuperar los datos que se encuentran almacenados en el navegador.

En JavaScript se utiliza el método getItem().

Ejemplo:

```
const usuario = localStorage.getItem("usuario");
```

Este procedimiento consulta el valor asociado con la clave usuario.

Cuando la información se encuentra guardada como JSON, es necesario convertirla nuevamente a un objeto o arreglo utilizando JSON.parse().

19.3. Escritura de información

La escritura permite guardar datos en el almacenamiento local.

Para ello se utiliza el método setItem().

Ejemplo:

```
localStorage.setItem("usuario", "Administrador");
```

Cuando se necesita almacenar un objeto o arreglo, primero debe convertirse a una cadena JSON.

```
const datos = {
```

```
  nombre: "Usuario de prueba",
  rol: "Instructor"
};

localStorage.setItem("usuario", JSON.stringify(datos));
```

Este procedimiento permite conservar estructuras de información que posteriormente pueden recuperarse.

19.4. Actualización de registros

La actualización consiste en recuperar la información existente, modificar los valores necesarios y guardar nuevamente el resultado.

En el caso de los arreglos, normalmente se utiliza JavaScript para localizar el registro correspondiente y actualizar sus propiedades.

Se recomienda comprobar que el registro exista antes de realizar la modificación y conservar los demás datos que no necesitan cambiarse.

19.5. Eliminación de registros

La eliminación permite retirar información del almacenamiento local.

El método removeItem() elimina el valor asociado con una clave específica.

Ejemplo:

```
localStorage.removeItem("usuario");
```

También existe el método clear(), que elimina todos los datos del almacenamiento local correspondiente al origen de la aplicación.

Se recomienda utilizar estas operaciones con precaución, especialmente cuando se manejan registros de inventario o usuarios.

19.6. Claves de almacenamiento utilizadas

Durante la revisión del proyecto se identificaron las siguientes claves:

Clave	Uso identificado
usuario	Información del usuario actual
usuarios	Registros utilizados en el proceso de registro
usuariosCentroSimulacion	Registros de administración de usuarios
inventarioProductos	Información de productos
movimientosInventario	Movimientos de inventario
reservasAmbiente	Información de reservas
notificacionesSENA	Notificaciones
preferenciasAjustesSENA	Preferencias de configuración
usosEquipos	Registros relacionados con el uso de equipos
tema-guia	Preferencia relacionada con el tema de las guías

Esta matriz debe actualizarse cuando se incorporen nuevas claves o se modifique la estructura de almacenamiento.

19.7. Serialización JSON

JSON es un formato utilizado para representar información estructurada.

En JavaScript se utiliza JSON.stringify() para convertir objetos en cadenas de texto y JSON.parse() para recuperar la estructura original.

Ejemplo:

```
const productos = [
  {
    nombre: "Guantes",
    cantidad: 50
  }
];

const texto = JSON.stringify(productos);
const resultado = JSON.parse(texto);
```

Es importante controlar los errores de conversión, ya que una cadena JSON incorrecta puede generar problemas durante la lectura.

19.8. Manejo de datos inconsistentes

Los datos inconsistentes son aquellos que presentan errores, información incompleta o estructuras diferentes a las esperadas por la aplicación.

Pueden aparecer cuando se modifican registros manualmente, se actualiza el código o se utilizan diferentes versiones del sistema.

Para reducir estos problemas se recomienda validar los datos al momento de leerlos y comprobar que tengan la estructura esperada.

También se deben realizar respaldos antes de modificar las claves de almacenamiento.

19.9. Copias de seguridad

Las copias de seguridad permiten conservar una copia de la información para recuperarla en caso de pérdida o modificación accidental.

Cuando se utiliza localStorage, los respaldos deben considerar las claves y los datos almacenados en cada navegador.

Se recomienda establecer procedimientos para exportar y conservar la información necesaria.

En un entorno institucional, la estrategia de respaldo debe definirse de acuerdo con las políticas de gestión documental y seguridad de la información.

19.10. Limitaciones del almacenamiento local

El almacenamiento local presenta limitaciones que deben tenerse en cuenta:

- Los datos se guardan en el navegador.
- No existe sincronización automática entre computadores.
- La información puede eliminarse al borrar los datos del navegador.
- No proporciona por sí mismo controles de acceso seguros.
- No reemplaza una base de datos centralizada.
- Los cambios realizados desde otro dispositivo no aparecen automáticamente.

Por estas razones, si el sistema se va a utilizar de manera simultánea por diferentes personas, se recomienda evaluar una arquitectura con backend y base de datos institucional.

CAPÍTULO 20. SEGURIDAD DEL FRONTEND

20.1. Principios de seguridad

La seguridad del frontend busca reducir los riesgos relacionados con la interacción de los usuarios con la aplicación.

Para ello se deben aplicar buenas prácticas en el manejo de formularios, la presentación de información, la navegación y el almacenamiento de datos.

También es importante comprender que el frontend no puede garantizar por sí solo la seguridad de toda la información. Los controles de autenticación, autorización y protección de datos sensibles deben complementarse con mecanismos implementados en el servidor.

20.2. Validación de entradas

La validación de entradas permite comprobar que los datos ingresados por los usuarios cumplan determinadas condiciones.

Se recomienda verificar los campos obligatorios, los formatos de correo electrónico, las cantidades y las fechas.

También es importante evitar que se procesen valores incorrectos que puedan afectar los registros.

La validación del navegador mejora la experiencia de uso, pero debe repetirse en el backend cuando exista una integración con servicios de datos.

20.3. Protección de rutas

La protección de rutas permite controlar qué páginas puede consultar un usuario según las condiciones establecidas por la aplicación.

En el proyecto se identifica el componente PrivateRoute, que participa en este proceso.

Es importante revisar las condiciones utilizadas y comprobar que las rutas restringidas se comporten correctamente.

Sin embargo, la protección visual de una ruta no impide que una persona manipule el código o las solicitudes desde el navegador. Por eso se requiere autorización del lado del servidor para proteger información sensible.

20.4. Manejo de credenciales

Las credenciales son los datos utilizados para identificar a una persona durante el proceso de autenticación.

Su manejo debe realizarse con precaución, evitando exponer contraseñas o información sensible en el código fuente, los mensajes de error o el almacenamiento local.

Se recomienda utilizar mecanismos seguros de autenticación y evitar guardar contraseñas en texto plano.

La implementación actual debe evaluarse antes de utilizarse como mecanismo definitivo de acceso institucional.

20.5. Riesgos del almacenamiento local

localStorage permite guardar información en el navegador, pero los datos pueden ser consultados o modificados desde las herramientas de desarrollo.

Esto significa que no debe utilizarse como único mecanismo para proteger información sensible o controlar permisos.

También existe el riesgo de pérdida de información cuando se eliminan los datos del navegador.

Se recomienda almacenar únicamente la información necesaria y evaluar mecanismos más seguros para los datos que requieran protección.

20.6. Control de acceso por roles

El control por roles permite organizar las funcionalidades según las responsabilidades de cada usuario.

En el sistema se identifican los perfiles Administrador, Instructor y Usuario invitado.

Es importante definir qué operaciones corresponden a cada perfil y comprobar que las restricciones se apliquen de manera coherente.

Los permisos deben validarse en el backend cuando se implemente una arquitectura centralizada.

20.7. Protección de información

La protección de información implica evitar el acceso, modificación o divulgación no autorizada de los datos del sistema.

Se recomienda no incluir información personal innecesaria en los reportes y evitar utilizar datos reales de pacientes en pruebas del frontend.

También se deben controlar los archivos exportados y los respaldos que contengan información institucional.

20.8. Recomendaciones de seguridad

Para fortalecer la seguridad del proyecto se recomienda:

- Mantener actualizadas las dependencias.
- Validar los datos ingresados.
- Evitar guardar contraseñas en texto plano.
- No incluir claves privadas en el código frontend.
- Implementar autenticación segura en backend.
- Establecer permisos en el servidor.
- Realizar respaldos periódicos.
- Revisar los errores y vulnerabilidades de las dependencias.
- Proteger los datos personales.
- Probar las rutas y formularios antes de publicar nuevas versiones.

20.9. Limitaciones de seguridad del frontend

El frontend se ejecuta en un entorno controlado por el navegador del usuario, por lo que su código y parte de su información pueden inspeccionarse.

Por esta razón, no debe utilizarse como única capa de protección para operaciones críticas.

El sistema requiere controles complementarios cuando se implemente una base de datos compartida o se manejen datos institucionales sensibles.

CAPÍTULO 21. PRUEBAS Y CONTROL DE CALIDAD

21.1. Conceptos de pruebas frontend

Las pruebas frontend permiten comprobar que la interfaz funcione correctamente y que las operaciones disponibles produzcan los resultados esperados.

Su objetivo es identificar errores antes de que las modificaciones sean utilizadas por los usuarios.

Las pruebas deben realizarse tanto en los componentes individuales como en las páginas y los recorridos completos de navegación.

21.2. Pruebas de componentes

Las pruebas de componentes permiten comprobar el comportamiento de elementos específicos.

Por ejemplo, se puede revisar si un botón ejecuta la acción correspondiente, si un formulario presenta sus campos correctamente o si una ventana se abre y se cierra de manera adecuada.

Se recomienda registrar el resultado de cada prueba y conservar evidencia de los errores encontrados.

21.3. Pruebas de navegación

Las pruebas de navegación permiten comprobar que las rutas conduzcan a las páginas correspondientes.

Se deben revisar las rutas públicas, las privadas, las redirecciones y el acceso a los diferentes módulos.

También es importante comprobar qué sucede cuando se ingresa una dirección que no se encuentra registrada.

21.4. Pruebas de formularios

Las pruebas de formularios permiten verificar que los datos se reciban y procesen correctamente.

Se recomienda realizar pruebas con información válida, campos vacíos, formatos incorrectos y valores fuera de los límites establecidos.

También se debe comprobar que los mensajes de error sean claros y permitan corregir los datos.

21.5. Pruebas de autenticación

Las pruebas de autenticación permiten comprobar el comportamiento de los procesos de acceso y salida del sistema.

Se deben revisar los datos ingresados, la información del usuario actual y las redirecciones posteriores al acceso.

También es importante comprobar el comportamiento cuando no existe información de usuario almacenada.

21.6. Pruebas de inventario

Las pruebas de inventario deben verificar el registro, consulta y actualización de los productos.

También deben incluir escenarios relacionados con cantidades, stock mínimo, fechas de vencimiento y ubicación.

Se recomienda comprobar que los movimientos se conserven y que los valores resultantes sean coherentes.

21.7. Pruebas de reservas

Las pruebas de reservas permiten revisar el registro y consulta de las actividades programadas.

Se deben comprobar las fechas, los horarios y los ambientes seleccionados.

Cuando el sistema incluya validaciones de disponibilidad, se deben probar los casos de coincidencia y los horarios libres.

21.8. Pruebas de reportes

Las pruebas de reportes permiten verificar que la información exportada corresponda con los datos disponibles.

Se recomienda comprobar la generación de documentos PDF, la presentación de tablas y la legibilidad del contenido.

También se deben revisar los casos donde no existen registros para evitar que se generen documentos vacíos o incorrectos.

21.9. Pruebas de compatibilidad

Las pruebas de compatibilidad permiten revisar el comportamiento del sistema en diferentes navegadores y tamaños de pantalla.

Se recomienda probar la aplicación en versiones actualizadas de los navegadores utilizados por la institución.

También se deben comprobar los formularios, tablas, imágenes y menús en diferentes resoluciones.

21.10. Registro de errores

El registro de errores permite documentar las fallas identificadas durante las pruebas.

Cada incidencia debería incluir:

- Código o número de identificación.
- Fecha de detección.
- Página o módulo afectado.
- Descripción del problema.
- Pasos para reproducirlo.
- Resultado esperado.
- Resultado obtenido.
- Estado de la corrección.

Este procedimiento facilita hacer seguimiento a los problemas y comprobar que hayan sido solucionados.

21.11. Criterios de aceptación

Los criterios de aceptación permiten determinar si una funcionalidad cumple las condiciones establecidas.

Una funcionalidad puede considerarse aceptada cuando realiza correctamente las operaciones previstas, presenta la información esperada y no genera errores que impidan su utilización.

Los criterios deben definirse antes de realizar las pruebas y deben corresponder con el alcance real del módulo.

CAPÍTULO 22. COMPILACIÓN Y DESPLIEGUE

22.1. Preparación para producción

Antes de publicar la aplicación se debe comprobar que el código se encuentre organizado y que las funcionalidades principales hayan sido probadas.

También es importante revisar las dependencias, los recursos gráficos y las configuraciones necesarias para el entorno de producción.

Se recomienda conservar una copia del proyecto antes de realizar la compilación definitiva.

22.2. Ejecución de npm run build

El comando npm run build permite ejecutar el proceso de compilación definido en el proyecto.

Para utilizarlo se debe abrir la terminal en la carpeta principal y ejecutar:

npm run build

Durante este proceso, Vite analiza los archivos y genera los recursos optimizados para su publicación.

Si se presentan errores, deben revisarse los mensajes de la terminal antes de continuar.

22.3. Carpeta dist

La carpeta dist contiene los archivos generados durante la compilación de producción.

Estos recursos son los que posteriormente pueden publicarse en un servidor web compatible.

No se recomienda realizar modificaciones manuales dentro de dist, ya que los cambios pueden perderse al ejecutar nuevamente la compilación.

22.4. Verificación de compilación

La verificación permite comprobar que la compilación haya terminado correctamente.

Se debe revisar que la carpeta dist exista y que contenga los archivos esperados.

También se recomienda ejecutar pruebas sobre la versión compilada para identificar problemas que no se hayan presentado durante el desarrollo.

22.5. Publicación del frontend

La publicación consiste en colocar los archivos de producción en un servidor o servicio de alojamiento compatible.

El procedimiento depende de la infraestructura definida por la institución.

Antes de realizar el despliegue se debe verificar la configuración de rutas, los recursos y las condiciones de acceso.

No debe considerarse una publicación institucional hasta que se haya autorizado el entorno correspondiente.

22.6. Configuración del servidor web

El servidor web debe configurarse para entregar los archivos estáticos generados por Vite.

Cuando se utiliza React Router DOM con navegación basada en historial, también puede ser necesario configurar el servidor para que las rutas de la aplicación sean atendidas correctamente.

La configuración específica dependerá del servidor utilizado y del entorno de publicación.

22.7. Verificación posterior al despliegue

Después de publicar la aplicación se deben realizar pruebas para comprobar que las páginas carguen correctamente.

Se recomienda verificar el acceso inicial, la navegación, los recursos gráficos, los formularios y las rutas principales.

También se deben revisar los errores de consola y confirmar que la aplicación funcione en el entorno donde fue publicada.

22.8. Actualización de versiones

La actualización consiste en reemplazar la versión publicada por una nueva versión de la aplicación.

Antes de actualizar se recomienda realizar una copia de seguridad, revisar los cambios y ejecutar las pruebas correspondientes.

También se debe comprobar que las dependencias sean compatibles y que los datos existentes no se vean afectados.

CAPÍTULO 23. MANTENIMIENTO Y SOLUCIÓN DE PROBLEMAS

23.1. Mantenimiento preventivo

El mantenimiento preventivo consiste en realizar revisiones periódicas para reducir la posibilidad de errores.

Dentro de estas actividades se encuentran la revisión de dependencias, la comprobación de rutas, la organización del código y la verificación de los recursos gráficos.

También es conveniente revisar que los datos almacenados mantengan una estructura coherente.

23.2. Mantenimiento correctivo

El mantenimiento correctivo se realiza cuando se identifica una falla en el sistema.

Su objetivo es encontrar la causa del problema y aplicar una modificación que permita recuperar el funcionamiento esperado.

Antes de corregir un error, se recomienda reproducirlo, identificar el archivo afectado y comprobar si existen otros componentes relacionados.

Después de la corrección deben realizarse pruebas para verificar que el problema haya sido solucionado.

23.3. Actualización de dependencias

Las dependencias deben mantenerse en versiones compatibles con el proyecto.

Antes de actualizar un paquete se recomienda revisar su documentación, los cambios introducidos y los posibles problemas de compatibilidad.

Es conveniente realizar las actualizaciones de manera controlada y evitar cambiar muchos paquetes al mismo tiempo sin realizar pruebas.

23.4. Errores frecuentes de React

Durante el desarrollo pueden presentarse errores relacionados con componentes, estados, propiedades y renderizado.

Algunos ejemplos son:

- Importaciones incorrectas.
- Componentes que no se exportan correctamente.
- Variables no definidas.
- Uso incorrecto de Hooks.
- Propiedades con valores inesperados.
- Claves faltantes en listas.
- Errores al procesar datos.

La revisión de la consola y del archivo señalado por el mensaje de error permite identificar la causa con mayor facilidad.

23.5. Errores de rutas

Los errores de rutas pueden producirse cuando una dirección no está definida, una página no se importa correctamente o una redirección se encuentra mal configurada.

Para solucionarlos se debe revisar App.jsx, comprobar las importaciones y verificar los componentes utilizados para controlar el acceso.

También se recomienda probar la dirección directamente en el navegador.

23.6. Problemas de almacenamiento

Los problemas de almacenamiento pueden presentarse cuando los datos tienen una estructura incorrecta, se utilizan claves diferentes o se intenta convertir una cadena JSON inválida.

Para solucionarlos se debe revisar el contenido de localStorage desde las herramientas de desarrollo del navegador.

No se recomienda eliminar todas las claves sin comprobar primero si contienen información necesaria.

23.7. Problemas de estilos

Los problemas visuales pueden estar relacionados con reglas CSS, nombres de clases, tamaños de pantalla o conflictos entre estilos.

Para identificarlos se pueden utilizar las herramientas de inspección del navegador.

Es importante revisar qué regla está afectando al elemento y comprobar que los cambios no alteren la presentación de otras páginas.

23.8. Diagnóstico desde consola

La consola del navegador permite observar mensajes y errores generados durante la ejecución de la aplicación.

Para abrirla se puede presionar F12 y seleccionar la pestaña Console.

Allí se pueden revisar errores de JavaScript, advertencias de React y otros mensajes útiles para el diagnóstico.

La terminal de Vite también debe revisarse, ya que puede mostrar errores relacionados con la compilación y las dependencias.

23.9. Procedimiento de recuperación

Cuando se presenta una falla importante se recomienda seguir un procedimiento ordenado:

1. Identificar el problema.

    Registrar el mensaje de error.

    Determinar el módulo afectado.

    Revisar los cambios recientes.

    Realizar una copia de seguridad.

    Corregir la causa identificada.

    Ejecutar nuevamente las pruebas.

    Documentar la solución aplicada.

Este procedimiento permite reducir modificaciones innecesarias y conservar evidencia de las acciones realizadas.

23.10. Recomendaciones de continuidad

Para mantener el proyecto en buenas condiciones se recomienda conservar copias de seguridad, documentar las modificaciones y utilizar un sistema de control de versiones.

También es importante que las personas encargadas del mantenimiento conozcan la estructura del proyecto y los mecanismos de almacenamiento utilizados.

Las futuras actualizaciones deben responder a necesidades identificadas y contar con pruebas antes de publicarse.

CAPÍTULO 24. ESTÁNDARES Y BUENAS PRÁCTICAS

24.1. Convenciones de nomenclatura

Las convenciones de nomenclatura permiten mantener nombres claros y consistentes en los archivos, componentes, variables y funciones.

Se recomienda utilizar nombres descriptivos que permitan comprender la responsabilidad de cada elemento.

Los componentes de React suelen nombrarse utilizando PascalCase, mientras que las variables y funciones pueden utilizar camelCase.

Mantener una convención uniforme facilita la lectura y el mantenimiento del código.

24.2. Organización del código

El código debe organizarse de manera que cada archivo tenga una función comprensible.

Se recomienda separar los componentes reutilizables de las páginas y evitar concentrar toda la lógica de la aplicación en un único archivo.

También es importante eliminar código que ya no se utiliza y mantener las importaciones organizadas.

24.3. Reutilización y modularidad

La reutilización permite aprovechar componentes existentes en diferentes partes del sistema.

La modularidad facilita dividir las funcionalidades en secciones independientes.

Ambas prácticas ayudan a reducir la duplicación y permiten realizar cambios de manera más controlada.

24.4. Accesibilidad

La accesibilidad debe tenerse en cuenta durante el desarrollo de los formularios, botones, menús y demás elementos de la interfaz.

Se recomienda utilizar etiquetas descriptivas, mantener contraste suficiente y permitir la navegación mediante teclado.

También es importante que los mensajes de error puedan comprenderse sin depender únicamente del color.

24.5. Rendimiento

El rendimiento está relacionado con el tiempo que tarda la aplicación en cargar y responder a las acciones del usuario.

Para mejorarlo se recomienda optimizar las imágenes, evitar cálculos innecesarios y mantener las dependencias bajo control.

También es importante revisar los componentes que producen actualizaciones frecuentes y comprobar que no se ejecuten operaciones innecesarias.

24.6. Calidad del código

La calidad del código permite que el proyecto sea comprensible, mantenible y menos propenso a errores.

Se recomienda utilizar nombres claros, mantener una estructura uniforme y revisar los cambios antes de integrarlos.

La ejecución de las herramientas de revisión configuradas en el proyecto permite identificar algunos problemas de manera anticipada.

24.7. Control de versiones

El control de versiones permite registrar la evolución del proyecto.

Git facilita identificar qué archivos fueron modificados y conservar versiones anteriores.

Se recomienda utilizar mensajes descriptivos para cada cambio y realizar copias de seguridad antes de modificaciones importantes.

24.8. Documentación técnica

La documentación técnica debe mantenerse actualizada y corresponder con el código fuente.

Cada modificación importante debería acompañarse de una explicación sobre su propósito y los archivos afectados.

También es conveniente registrar los cambios en las dependencias, las rutas y las estructuras de almacenamiento.

24.9. Recomendaciones para futuras versiones

Para futuras versiones del sistema se recomienda evaluar la implementación de un backend, una base de datos centralizada y mecanismos seguros de autenticación.

También pueden incorporarse mejoras al asistente virtual, controles adicionales de acceso y herramientas de seguimiento de los recursos.

Estas mejoras deben planificarse según las necesidades institucionales y contar con una evaluación técnica antes de implementarse.

CAPÍTULO 25. ANEXOS

25.1. Glosario técnico ampliado

Este glosario reúne los conceptos más importantes utilizados durante el desarrollo y mantenimiento del frontend.

Término	Definición
React	Biblioteca de JavaScript para construir interfaces de usuario.
Frontend	Parte de una aplicación con la que interactúa directamente el usuario.
Backend	Parte encargada de procesar operaciones y proporcionar servicios al frontend.
Componente	Unidad reutilizable de una interfaz React.
JSX	Extensión de sintaxis que permite describir interfaces dentro de JavaScript.
Props	Propiedades utilizadas para transmitir información entre componentes.
State	Información que puede cambiar durante la ejecución de un componente.
Hook	Función de React que permite utilizar determinadas características de la biblioteca.
useState	Hook utilizado para gestionar estados.
useEffect	Hook utilizado para gestionar efectos secundarios.
Context API	Mecanismo para compartir información entre componentes.
Vite	Herramienta de desarrollo y compilación frontend.
React Router DOM	Biblioteca para gestionar la navegación.
localStorage	Almacenamiento local del navegador.
JSON	Formato de representación de datos estructurados.
npm	Administrador de paquetes utilizado en proyectos JavaScript.
Node.js	Entorno de ejecución de JavaScript.
DOM	Representación estructurada del documento web.
CSS	Lenguaje utilizado para definir la presentación visual.
HTML	Lenguaje utilizado para estructurar contenidos web.
Build	Proceso de preparación de una versión de producción.
Deploy	Proceso de publicación de una aplicación.
Debugging	Proceso de identificación y corrección de errores.
Git	Sistema de control de versiones.

25.2. Inventario de componentes

El siguiente inventario reúne los componentes identificados en el proyecto y permite documentar su responsabilidad general.

Componente	Función
Sidebar	Presentación del menú lateral.
Navbar	Presentación de la barra superior.
PrivateRoute	Control de acceso a rutas.
NotificationPanel	Presentación de notificaciones.
IAButton	Acceso a la interfaz del asistente virtual.
DashboardLayout	Organización del entorno principal.
AuthContext	Administración del contexto de autenticación.

Este inventario debe ampliarse cuando se identifiquen otros componentes reutilizables.

25.3. Inventario de páginas

El proyecto contiene diferentes páginas que representan los módulos y funcionalidades disponibles.

Entre ellas se encuentran:

Página	Descripción
Login	Inicio de sesión.
Registro	Registro de usuarios.
Recuperar	Recuperación de acceso.
Dashboard	Panel principal.
Ambientes	Gestión de ambientes.
Equipos	Gestión de equipos.
Inventario	Gestión de productos e insumos.
Cronograma	Consulta de actividades programadas.
Evaluaciones	Funcionalidades de evaluación.
Guias	Consulta de guías.
Notificaciones	Consulta de notificaciones.
Miperfil	Información del perfil.
Reportes	Presentación de reportes.
Usuarios	Administración de usuarios.
Simulaciones	Funcionalidades de simulación.

25.4. Matriz de rutas

La matriz de rutas permite identificar las direcciones que utiliza la aplicación para acceder a las diferentes páginas.

Dirección	Página
/	Página inicial
/login	Inicio de sesión
/registro	Registro
/crear-cuenta	Creación de cuenta
/recuperar	Recuperación
/acerca	Información general
/contacto	Contacto
/servicios	Servicios
/dashboard	Panel principal
/ambientes	Ambientes
/equipos	Equipos
/inventario	Inventario
/estado-inventario	Estado del inventario
/registro-inventario	Registro de inventario
/cronograma	Cronograma
/evaluaciones	Evaluaciones
/guias	Guías
/notificaciones	Notificaciones
/mi-perfil	Perfil
/perfil	Perfil
/reportes	Reportes
/ajustes	Ajustes
/simulaciones	Simulaciones
/usuarios	Usuarios

La protección de cada dirección debe verificarse en la configuración de rutas.

25.5. Matriz de almacenamiento local

Esta matriz permite identificar las claves que utiliza el sistema para conservar información en el navegador.

Clave	Información
usuario	Usuario actual.
usuarios	Registros del proceso de registro.
usuariosCentroSimulacion	Información de administración de usuarios.
inventarioProductos	Productos del inventario.
movimientosInventario	Movimientos de productos.
reservasAmbiente	Reservas de ambientes.
notificacionesSENA	Notificaciones.
preferenciasAjustesSENA	Preferencias de configuración.
usosEquipos	Registros de uso de equipos.
tema-guia	Preferencia del tema de guías.

25.6. Matriz de pruebas

La matriz de pruebas permite registrar los resultados obtenidos durante la revisión de las funcionalidades.

Código	Módulo	Prueba	Resultado
PT-001	Acceso	Ingresar al sistema	Pendiente de ejecución
PT-002	Usuarios	Registrar usuario	Pendiente de ejecución
PT-003	Inventario	Registrar insumo	Pendiente de ejecución
PT-004	Inventario	Actualizar existencias	Pendiente de ejecución
PT-005	Ambientes	Consultar ambientes	Pendiente de ejecución
PT-006	Reservas	Registrar reserva	Pendiente de ejecución
PT-007	Equipos	Consultar equipos	Pendiente de ejecución
PT-008	Reportes	Generar PDF	Pendiente de ejecución
PT-009	Navegación	Acceder a los módulos	Pendiente de ejecución
PT-010	Notificaciones	Consultar mensajes	Pendiente de ejecución

Los resultados deben completarse después de ejecutar las pruebas. No se deben marcar como aprobadas sin evidencia.

25.7. Formato de registro de incidencias

El registro de incidencias permite documentar los problemas encontrados durante el desarrollo o funcionamiento del sistema.

Campo	Información
Código	Identificador de la incidencia
Fecha	Día en que se identificó
Módulo	Sección afectada
Descripción	Explicación del problema
Evidencia	Captura o mensaje de error
Responsable	Persona encargada de revisar
Solución	Acción aplicada
Estado	Pendiente, en proceso o solucionada

Este formato facilita el seguimiento y permite conservar un historial de los problemas encontrados.

25.8. Formato de control de cambios

El control de cambios permite registrar las modificaciones realizadas en el código fuente.

Versión	Fecha	Archivo	Modificación	Responsable
1.0	Por definir	Proyecto completo	Versión inicial documentada	Por definir
1.1	Por definir	Por definir	Mejora o corrección	Por definir

El historial debe actualizarse cada vez que se realicen modificaciones importantes.

25.9. Referencias técnicas

React. (s. f.). React documentation. https://react.dev/

Vite. (s. f.). Vite documentation. https://vite.dev/

React Router. (s. f.). React Router documentation. https://reactrouter.com/

MDN Web Docs. (s. f.). JavaScript. https://developer.mozilla.org/es/docs/Web/JavaScript

MDN Web Docs. (s. f.). Web Storage API. https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

Node.js. (s. f.). Node.js documentation. https://nodejs.org/docs/

npm. (s. f.). npm documentation. https://docs.npmjs.com/

25.10. Historial de versiones del manual

Versión	Fecha	Descripción	Responsable
1.0	Por definir	Elaboración inicial del manual técnico	Por definir
1.1	Por definir	Actualización de contenido técnico	Por definir
1.2	Por definir	Inclusión de nuevas funcionalidades y capturas	Por definir

Capítulo	Evidencia que debes agregar
1	Captura del sistema y descripción del propósito.
2	Diagrama de tecnologías utilizadas.
3	Capturas de Node.js, VS Code y terminal.
4	Capturas de instalación, package.json y ejecución.
5	Captura de carpetas y archivos del proyecto.
6	Ejemplos de componentes reales del código.
7	Fragmentos de AuthContext.jsx.
8	Captura de App.jsx y diagrama de rutas.
9	Capturas de dashboard, Sidebar y Navbar.
10	Capturas de login, registro y usuarios.
11	Capturas del módulo de ambientes.
12	Capturas del módulo de equipos.
13	Capturas del inventario y ejemplos de registros.
14	Capturas del cronograma y reservas.
15	Capturas de simulaciones y prácticas.
16	Capturas de guías y evaluaciones.
17	Ejemplos de reportes y notificaciones.
18	Capturas del asistente y explicación de su código.
19	Ejemplos de lectura y escritura de localStorage.
20	Matriz de riesgos y recomendaciones de seguridad.
21	Casos de prueba con evidencias.
22	Capturas de compilación y carpeta dist.
23	Tabla de errores frecuentes y soluciones.
24	Lista de buenas prácticas y estándares.
25	Matrices, glosario y formatos de control.